

./embedder_core/init.py

```
"""Standalone embedder package."""

from .cli import main
from .constants import MASK, PAD, SPECIAL_TOKENS, UNK
from .data import AugmentConfig, KeystrokeUserDataset, collate_fn
from .model import KeystrokeTransformer
from .tokens import KeypressToken, parse_run_to_tokens
from .train import TrainConfig, evaluate, load_model_weights, train
from .utils import accuracy_from_logits, cosine_with_warmup, set_seed
from .visualize import save_pca_embedding_plot
from .vocab import build_keysym_vocab, normalize_keysym

__all__ = [
    "PAD",
    "UNK",
    "MASK",
    "SPECIAL_TOKENS",
    "KeypressToken",
    "AugmentConfig",
    "TrainConfig",
    "KeystrokeUserDataset",
    "KeystrokeTransformer",
    "accuracy_from_logits",
    "build_keysym_vocab",
    "collate_fn",
    "cosine_with_warmup",
    "evaluate",
    "load_model_weights",
    "main",
    "normalize_keysym",
    "parse_run_to_tokens",
    "save_pca_embedding_plot",
    "set_seed",
    "train",
]
```

./embedder_core/cli.py

```
import argparse
import sys
from typing import List, Optional

from .train import train

def main(argv: Optional[List[str]] = None) -> int:
    args_in = list(sys.argv[1:] if argv is None else argv)

    parser = argparse.ArgumentParser(description="Train the keystroke embedder.")
    parser.add_argument("json_paths", nargs="+", help="User JSON files (one file per class/user).")
    parser.add_argument("--min-freq", type=int, default=2, help="Minimum token frequency for vocab incl")
```

```

parser.add_argument(
    "--load-weights",
    default=None,
    help="Optional path to an existing weights/checkpoint file to initialize the model from.",
)
parser.add_argument("--save-path", default=None, help="Optional path for best-checkpoint output.")
parser.add_argument("--final-weights-path", default=None, help="Optional path for final weights output.")
parser.add_argument("--pca-plot-path", default=None, help="Optional path for PCA plot output.")
args = parser.parse_args(args_in)

paths = list(args.json_paths)
if len(paths) < 2:
    print(
        "Pass >=2 user json paths (one per person). "
        "Example: python embedder.py u0.json u1.json u2.json u3.json"
    )
    return 1

train(
    paths,
    min_freq=args.min_freq,
    load_weights_path=args.load_weights,
    save_path=args.save_path,
    final_weights_path=args.final_weights_path,
    pca_plot_path=args.pca_plot_path,
)
return 0

```

./embedder_core/constants.py

```

PAD = "<PAD>"
UNK = "<UNK>"
MASK = "<MASK>"

```

```
SPECIAL_TOKENS = [PAD, UNK, MASK]
```

./embedder_core/data.py

```

import json
import random
from dataclasses import dataclass
from typing import Any, Dict, List, Tuple

import torch
from torch.utils.data import Dataset

from .tokens import KeypressToken, parse_run_to_tokens

class KeystrokeUserDataset(Dataset):
    """
    Expects one JSON per user:
    { "enrollment_runs": [ {"events": [...]}, ... ] }
    """

```

```

"""

def __init__(
    self,
    user_json_paths: List[str],
    stoi: Dict[str, int],
    split: str = "train",
    val_frac: float = 0.20,
    seed: int = 42,
    max_tokens: int = 256,
) -> None:
    super().__init__()
    assert split in ("train", "val", "all")
    self.split = split
    self.max_tokens = max_tokens
    self.stoi = stoi

    all_samples: List[Tuple[List[KeypressToken], int]] = []
    for label, path in enumerate(user_json_paths):
        with open(path, "r", encoding="utf-8") as handle:
            blob = json.load(handle)
            runs = blob.get("enrollment_runs", [])
            for run in runs:
                tokens = parse_run_to_tokens(run.get("events", []), stoi)
                if len(tokens) < 5:
                    continue
                all_samples.append((tokens, label))

    rng = random.Random(seed)
    by_label: Dict[int, List[Tuple[List[KeypressToken], int]]] = {}
    for sample in all_samples:
        by_label.setdefault(sample[1], []).append(sample)

    train_samples: List[Tuple[List[KeypressToken], int]] = []
    val_samples: List[Tuple[List[KeypressToken], int]] = []
    for samples in by_label.values():
        rng.shuffle(samples)
        n_val = max(1, int(len(samples) * val_frac))
        val_samples.extend(samples[:n_val])
        train_samples.extend(samples[n_val:])

    if split == "train":
        self.samples = train_samples
    elif split == "val":
        self.samples = val_samples
    else:
        self.samples = all_samples

    rng.shuffle(self.samples)

def __len__(self) -> int:
    return len(self.samples)

def __getitem__(self, idx: int) -> Dict[str, Any]:

```

```

tokens, label = self.samples[idx]
if len(tokens) > self.max_tokens:
    tokens = tokens[: self.max_tokens]

keysym_ids = torch.tensor([t.keysym_id for t in tokens], dtype=torch.long)
dwell = torch.tensor([t.dwell_ms for t in tokens], dtype=torch.float32)
flight = torch.tensor([t.flight_ms for t in tokens], dtype=torch.float32)
y = torch.tensor(label, dtype=torch.long)
return {"keysym_ids": keysym_ids, "dwell": dwell, "flight": flight, "y": y}

@dataclass
class AugmentConfig:
    enable: bool = True
    speed_scale_min: float = 0.9
    speed_scale_max: float = 1.1
    jitter_std_ms: float = 10.0
    token_drop_prob: float = 0.06
    key_mask_prob: float = 0.15
    key_unk_prob: float = 0.03
    clip_ms: float = 800.0
    log1p: bool = True

def collate_fn(
    batch: List[Dict[str, Any]],
    aug: AugmentConfig,
    is_train: bool,
    pad_id: int,
    mask_id: int,
    unk_id: int,
) -> Dict[str, torch.Tensor]:
    ks_seqs: List[torch.Tensor] = []
    dwell_seqs: List[torch.Tensor] = []
    flight_seqs: List[torch.Tensor] = []
    ys: List[torch.Tensor] = []

    for ex in batch:
        ks = ex["keysym_ids"].clone()
        dwell = ex["dwell"].clone()
        flight = ex["flight"].clone()
        y = ex["y"]

        if is_train and aug.enable:
            if len(ks) > 8 and aug.token_drop_prob > 0:
                keep = torch.rand(len(ks)) > aug.token_drop_prob
                if keep.sum().item() >= 5:
                    ks = ks[keep]
                    dwell = dwell[keep]
                    flight = flight[keep]

            if aug.key_mask_prob > 0:
                mask = torch.rand(len(ks)) < aug.key_mask_prob
                ks[mask] = mask_id

```

```

        if aug.key_unk_prob > 0:
            mask = torch.rand(len(ks)) < aug.key_unk_prob
            ks[mask] = unk_id

        scale = random.uniform(aug.speed_scale_min, aug.speed_scale_max)
        dwell = dwell * scale
        flight = flight * scale

        if aug.jitter_std_ms > 0:
            dwell = dwell + torch.randn_like(dwell) * aug.jitter_std_ms
            flight = flight + torch.randn_like(flight) * aug.jitter_std_ms

        dwell = torch.clamp(dwell, 0.0, aug.clip_ms)
        flight = torch.clamp(flight, 0.0, aug.clip_ms)

        if aug.log1p:
            dwell = torch.log1p(dwell)
            flight = torch.log1p(flight)

        ks_seqs.append(ks)
        dwell_seqs.append(dwell)
        flight_seqs.append(flight)
        ys.append(y)

    lengths = torch.tensor([len(seq) for seq in ks_seqs], dtype=torch.long)
    max_len = int(lengths.max().item())

    ks_pad = torch.full((len(batch), max_len), pad_id, dtype=torch.long)
    dwell_pad = torch.zeros((len(batch), max_len), dtype=torch.float32)
    flight_pad = torch.zeros((len(batch), max_len), dtype=torch.float32)
    attn_mask = torch.zeros((len(batch), max_len), dtype=torch.bool)

    for i, (ks, dwell, flight) in enumerate(zip(ks_seqs, dwell_seqs, flight_seqs)):
        seq_len = len(ks)
        ks_pad[i, :seq_len] = ks
        dwell_pad[i, :seq_len] = dwell
        flight_pad[i, :seq_len] = flight
        attn_mask[i, :seq_len] = True

    y = torch.stack(ys)
    return {
        "keysym_ids": ks_pad,
        "dwell": dwell_pad,
        "flight": flight_pad,
        "attn_mask": attn_mask,
        "y": y,
        "lengths": lengths,
    }
}

```

./embedder_core/model.py

```

import torch
import torch.nn as nn

```

```

class KeystrokeTransformer(nn.Module):
    def __init__(
        self,
        num_classes: int,
        vocab_size: int,
        d_model: int = 160,
        nhead: int = 4,
        num_layers: int = 3,
        dim_ff: int = 320,
        dropout: float = 0.1,
        use_cls_token: bool = True,
    ) -> None:
        super().__init__()
        self.use_cls_token = use_cls_token
        self.d_model = d_model

        self.key_emb = nn.Embedding(vocab_size, d_model)
        self.time_mlp = nn.Sequential(
            nn.Linear(2, d_model),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(d_model, d_model),
        )
        self.ln = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(dropout)

        if use_cls_token:
            self.cls = nn.Parameter(torch.zeros(1, 1, d_model))
        else:
            self.cls = None

        enc_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_ff,
            dropout=dropout,
            batch_first=True,
            activation="gelu",
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(enc_layer, num_layers=num_layers)

        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, num_classes),
        )

        nn.init.normal_(self.key_emb.weight, mean=0.0, std=0.02)
        if self.cls is not None:
            nn.init.normal_(self.cls, mean=0.0, std=0.02)

    def encode(

```

```

        self,
        keysym_ids: torch.Tensor,
        dwell: torch.Tensor,
        flight: torch.Tensor,
        attn_mask: torch.Tensor,
    ) -> torch.Tensor:
        batch_size, _ = keysym_ids.shape

        key_features = self.key_emb(keysym_ids)
        time_features = torch.stack([dwell, flight], dim=-1)
        time_embeddings = self.time_mlp(time_features)

        x = self.ln(key_features + time_embeddings)
        x = self.drop(x)

        if self.use_cls_token:
            cls = self.cls.expand(batch_size, 1, self.d_model)
            x = torch.cat([cls, x], dim=1)
            attn_mask = torch.cat(
                [torch.ones(batch_size, 1, device=attn_mask.device, dtype=torch.bool), attn_mask],
                dim=1,
            )

        src_key_padding_mask = ~attn_mask
        hidden = self.encoder(x, src_key_padding_mask=src_key_padding_mask)

        if self.use_cls_token:
            pooled = hidden[:, 0, :]
        else:
            mask = attn_mask.unsqueeze(-1).float()
            pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp_min(1.0)

        return pooled

    def forward(
        self,
        keysym_ids: torch.Tensor,
        dwell: torch.Tensor,
        flight: torch.Tensor,
        attn_mask: torch.Tensor,
    ) -> torch.Tensor:
        pooled = self.encode(keysym_ids, dwell, flight, attn_mask)
        return self.head(pooled)

```

./embedder_core/tokens.py

```

from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from .constants import UNK
from .vocab import normalize_keysym

```

```

@dataclass
class KeypressToken:
    keySYM_id: int
    dwell_ms: float
    flight_ms: float

def parse_run_to_tokens(events: List[Dict[str, Any]], stoi: Dict[str, int]) -> List[KeyPressToken]:
    """
    Pair keydown->keyup by keycode. Emit keypress tokens ordered by keydown time.

    Token identity uses normalize_keySYM on the keyup event (or falls back).
    """
    down_time: Dict[int, float] = {}
    down_keySYM: Dict[int, str] = {}

    keypresses: List[Tuple[float, int, float, int]] = []
    # (down_ts, keycode, up_ts, keySYM_id)

    for ev in events:
        ev_type = ev.get("type")
        ts = float(ev.get("timestamp_ms"))
        keycode = ev.get("keycode")
        if keycode is None:
            continue
        keycode = int(keycode)

        if ev_type == "keydown":
            down_time[keycode] = ts
            down_keySYM[keycode] = normalize_keySYM(ev)
        elif ev_type == "keyup":
            if keycode not in down_time:
                continue
            down_ts = down_time.pop(keycode)
            keySYM = normalize_keySYM(ev)
            if keySYM == UNK and keycode in down_keySYM:
                keySYM = down_keySYM[keycode]

            keySYM_id = stoi.get(keySYM, stoi[UNK])
            keypresses.append((down_ts, keycode, ts, keySYM_id))
            down_keySYM.pop(keycode, None)

    keypresses.sort(key=lambda row: row[0])

    tokens: List[KeyPressToken] = []
    prev_down: Optional[float] = None
    for down_ts, _, up_ts, keySYM_id in keypresses:
        dwell = max(0.0, up_ts - down_ts)
        flight = 0.0 if prev_down is None else max(0.0, down_ts - prev_down)
        prev_down = down_ts
        tokens.append(KeypressToken(keySYM_id, dwell, flight))

    return tokens

```


./embedder_core/train.py

```
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

from .constants import MASK, PAD, UNK
from .data import AugmentConfig, KeystrokeUserDataset, collate_fn
from .model import KeystrokeTransformer
from .utils import accuracy_from_logits, cosine_with_warmup, set_seed
from .visualize import save_pca_embedding_plot
from .vocab import build_keysym_vocab

@dataclass
class TrainConfig:
    epochs: int = 0
    batch_size: int = 32
    lr: float = 2e-4
    weight_decay: float = 0.01
    grad_clip: float = 1.0
    warmup_steps: int = 100
    val_frac: float = 0.15
    max_tokens: int = 256
    num_workers: int = 0
    seed: int = 42
    device: str = "cuda" if torch.cuda.is_available() else "cpu"
    early_stop_patience: int = 8
    save_path: str = "keystroke_user_classifier_keysym.pt"
    final_weights_path: str = "keystroke_user_classifier_keysym_final_weights.pt"
    pca_plot_path: str = "keystroke_user_classifier_keysym_pca.png"

@torch.no_grad()
def evaluate(model: nn.Module, loader: DataLoader, device: str) -> Tuple[float, float]:
    model.eval()
    criterion = nn.CrossEntropyLoss()
    losses = []
    accuracies = []
    for batch in loader:
        for key in ("keysym_ids", "dwell", "flight", "attn_mask", "y"):
            batch[key] = batch[key].to(device)
        logits = model(batch["keysym_ids"], batch["dwell"], batch["flight"], batch["attn_mask"])
        loss = criterion(logits, batch["y"])
        losses.append(loss.item())
        accuracies.append(accuracy_from_logits(logits, batch["y"]))
    return float(np.mean(losses)), float(np.mean(accuracies))

def _extract_state_dict(payload: Any) -> Dict[str, torch.Tensor]:
```

```

if isinstance(payload, dict) and isinstance(payload.get("model_state"), dict):
    return payload["model_state"]
if isinstance(payload, dict) and all(isinstance(v, torch.Tensor) for v in payload.values()):
    return payload
raise ValueError("Weights file must be a torch state_dict or checkpoint with a 'model_state' key.")

def load_model_weights(model: nn.Module, path: str, device: str) -> None:
    payload = torch.load(path, map_location=device)
    state_dict = _extract_state_dict(payload)
    missing, unexpected = model.load_state_dict(state_dict, strict=False)
    if not missing and not unexpected:
        print(f"Loaded weights from {path}")
    else:
        print(
            f"Loaded weights from {path} with partial match "
            f"(missing={len(missing)}, unexpected={len(unexpected)})"
        )

def train(
    user_json_paths: List[str],
    min_freq: int = 2,
    *,
    load_weights_path: Optional[str] = None,
    save_path: Optional[str] = None,
    final_weights_path: Optional[str] = None,
    pca_plot_path: Optional[str] = None,
) -> None:
    cfg = TrainConfig()
    if save_path is not None:
        cfg.save_path = save_path
    if final_weights_path is not None:
        cfg.final_weights_path = final_weights_path
    if pca_plot_path is not None:
        cfg.pca_plot_path = pca_plot_path

    aug = AugmentConfig()
    set_seed(cfg.seed)

    stoi, itos = build_keysym_vocab(user_json_paths, min_freq=min_freq)
    pad_id = stoi[PAD]
    unk_id = stoi[UNK]
    mask_id = stoi[MASK]

    train_ds = KeystrokeUserDataset(
        user_json_paths=user_json_paths,
        stoi=stoi,
        split="train",
        val_frac=cfg.val_frac,
        seed=cfg.seed,
        max_tokens=cfg.max_tokens,
    )
    val_ds = KeystrokeUserDataset(

```

```

        user_json_paths=user_json_paths,
        stoi=stoi,
        split="val",
        val_frac=cfg.val_frac,
        seed=cfg.seed,
        max_tokens=cfg.max_tokens,
    )

    train_loader = DataLoader(
        train_ds,
        batch_size=cfg.batch_size,
        shuffle=True,
        num_workers=cfg.num_workers,
        collate_fn=lambda batch: collate_fn(batch, aug, True, pad_id, mask_id, unk_id),
    )
    val_loader = DataLoader(
        val_ds,
        batch_size=cfg.batch_size,
        shuffle=False,
        num_workers=cfg.num_workers,
        collate_fn=lambda batch: collate_fn(batch, aug, False, pad_id, mask_id, unk_id),
    )

    model = KeystrokeTransformer(
        num_classes=len(user_json_paths),
        vocab_size=len(itos),
        d_model=160,
        nhead=4,
        num_layers=3,
        dim_ff=320,
        dropout=0.1,
        use_cls_token=True,
    ).to(cfg.device)
    if load_weights_path:
        load_model_weights(model, path=load_weights_path, device=cfg.device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=cfg.lr, weight_decay=cfg.weight_decay)
    criterion = nn.CrossEntropyLoss()

    total_steps = cfg.epochs * max(1, len(train_loader))
    step = 0

    best_val_acc = -1.0
    patience = 0
    best_model_state = None

    for epoch in range(1, cfg.epochs + 1):
        model.train()
        epoch_losses = []
        epoch_accuracies = []

        for batch in train_loader:
            for key in ("keySYM_ids", "dwell", "flight", "attn_mask", "y"):
                batch[key] = batch[key].to(cfg.device)

```

```

lr_scale = cosine_with_warmup(step, total_steps, cfg.warmup_steps)
for group in optimizer.param_groups:
    group["lr"] = cfg.lr * lr_scale

logits = model(batch["keysym_ids"], batch["dwell"], batch["flight"], batch["attn_mask"])
loss = criterion(logits, batch["y"])

optimizer.zero_grad(set_to_none=True)
loss.backward()
if cfg.grad_clip > 0:
    torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
optimizer.step()

epoch_losses.append(loss.item())
epoch_accuracies.append(accuracy_from_logits(logits, batch["y"]))
step += 1

train_loss = float(np.mean(epoch_losses))
train_acc = float(np.mean(epoch_accuracies))
val_loss, val_acc = evaluate(model, val_loader, cfg.device)

print(
    f"Epoch {epoch:02d} | train loss {train_loss:.4f} acc {train_acc:.3f} "
    f"| val loss {val_loss:.4f} acc {val_acc:.3f}"
)

if val_acc > best_val_acc + 1e-4:
    best_val_acc = val_acc
    patience = 0
    best_model_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
    torch.save(
        {
            "model_state": model.state_dict(),
            "stoi": stoi,
            "itos": itos,
            "train_cfg": cfg.__dict__,
            "aug_cfg": aug.__dict__,
            "num_classes": len(user_json_paths),
        },
        cfg.save_path,
    )
    print(f" saved best -> {cfg.save_path} (val_acc={best_val_acc:.3f})")
else:
    patience += 1
    if patience >= cfg.early_stop_patience:
        print("Early stopping.")
        break

torch.save(
    {
        "model_state": model.state_dict(),
        "stoi": stoi,
        "itos": itos,
    },

```

```

        "train_cfg": cfg.__dict__,
        "aug_cfg": aug.__dict__,
        "num_classes": len(user_json_paths),
    },
    cfg.final_weights_path,
)
print(f"Saved final weights -> {cfg.final_weights_path}")

if best_model_state is not None:
    model.load_state_dict(best_model_state)

save_pca_embedding_plot(
    model=model,
    user_json_paths=user_json_paths,
    stoi=stoi,
    cfg=cfg,
    pad_id=pad_id,
    mask_id=mask_id,
    unk_id=unk_id,
    output_path=cfg.pca_plot_path,
)

```

./embedder_core/utils.py

```

import math
import random

import numpy as np
import torch

def set_seed(seed: int = 42) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

def accuracy_from_logits(logits: torch.Tensor, y: torch.Tensor) -> float:
    preds = torch.argmax(logits, dim=-1)
    return (preds == y).float().mean().item()

def cosine_with_warmup(step: int, total: int, warmup: int) -> float:
    if step < warmup:
        return float(step) / float(max(1, warmup))
    progress = float(step - warmup) / float(max(1, total - warmup))
    return 0.5 * (1.0 + math.cos(math.pi * progress))

```

./embedder_core/visualize.py

```

import os
from typing import Any, Dict, List

```

```

import numpy as np
import torch
from torch.utils.data import DataLoader

from .data import AugmentConfig, KeystrokeUserDataset, collate_fn
from .model import KeystrokeTransformer

@torch.no_grad()
def save_pca_embedding_plot(
    model: KeystrokeTransformer,
    user_json_paths: List[str],
    stoi: Dict[str, int],
    cfg: Any,
    pad_id: int,
    mask_id: int,
    unk_id: int,
    output_path: str,
) -> None:
    try:
        import matplotlib

        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from sklearn.decomposition import PCA
    except Exception as exc:
        print(f"Skipping PCA plot (missing matplotlib/scikit-learn): {exc}")
        return

    aug = AugmentConfig(enable=False)
    dataset = KeystrokeUserDataset(
        user_json_paths=user_json_paths,
        stoi=stoi,
        split="all",
        val_frac=cfg.val_frac,
        seed=cfg.seed,
        max_tokens=cfg.max_tokens,
    )
    if len(dataset) == 0:
        print("Skipping PCA plot: no usable samples.")
        return

    loader = DataLoader(
        dataset,
        batch_size=cfg.batch_size,
        shuffle=False,
        num_workers=cfg.num_workers,
        collate_fn=lambda batch: collate_fn(batch, aug, False, pad_id, mask_id, unk_id),
    )

    model.eval()
    all_embeddings = []
    all_labels = []

```

```

for batch in loader:
    for key in ("keysym_ids", "dwell", "flight", "attn_mask"):
        batch[key] = batch[key].to(cfg.device)
    emb = model.encode(batch["keysym_ids"], batch["dwell"], batch["flight"], batch["attn_mask"])
    all_embeddings.append(emb.detach().cpu().numpy())
    all_labels.append(batch["y"].detach().cpu().numpy())

embeddings = np.concatenate(all_embeddings, axis=0)
labels = np.concatenate(all_labels, axis=0)
if embeddings.shape[0] < 2:
    print("Skipping PCA plot: need at least 2 embeddings.")
    return

pca = PCA(n_components=2, random_state=cfg.seed)
points = pca.fit_transform(embeddings)
explained = pca.explained_variance_ratio_

num_users = len(user_json_paths)
if num_users <= 20:
    colors = plt.cm.tab20(np.linspace(0.0, 1.0, num_users))
else:
    colors = plt.cm.nipy_spectral(np.linspace(0.0, 1.0, num_users))

fig, ax = plt.subplots(figsize=(9, 7))
for user_idx, json_path in enumerate(user_json_paths):
    mask = labels == user_idx
    if not np.any(mask):
        continue
    ax.scatter(
        points[mask, 0],
        points[mask, 1],
        s=24,
        alpha=0.8,
        color=colors[user_idx],
        label=os.path.basename(json_path),
        edgecolors="none",
    )

ax.set_title("Keystroke Embeddings PCA")
ax.set_xlabel(f"PC1 ({explained[0] * 100:.1f}% var)")
ax.set_ylabel(f"PC2 ({explained[1] * 100:.1f}% var)")
ax.legend(loc="best", fontsize=8, frameon=False)
ax.grid(alpha=0.2)
fig.tight_layout()
fig.savefig(output_path, dpi=200)
plt.close(fig)
print(f"Saved PCA embedding plot -> {output_path}")

```

`./embedder_core/vocab.py`

```

import json
from typing import Any, Dict, List, Tuple

```

```

from .constants import SPECIAL_TOKENS, UNK

def normalize_keysym(ev: Dict[str, Any]) -> str:
    """
    Turn an event into a stable keysym string.

    Priorities:
    1) ev["keysym"] if present
    2) ev["char"] (single char)
    3) ev["keycode"] as "kc_XX"
    """
    keysym = ev.get("keysym")
    ch = ev.get("char")
    keycode = ev.get("keycode")

    if keysym is not None and str(keysym).strip() != "":
        s = str(keysym).strip().lower()
        if s in ("return",):
            s = "enter"
        if s in ("space",):
            s = "space"
        return s

    if ch is not None and str(ch) != "":
        s = str(ch)
        if len(s) == 1:
            return s.lower()
        return s.lower()

    if keycode is not None:
        try:
            return f"kc_{int(keycode)}"
        except Exception:
            pass

    return UNK

def build_keysym_vocab(user_json_paths: List[str], min_freq: int = 2) -> Tuple[Dict[str, int], List[str]]:
    """
    Scan all events across all users and build a keysym vocabulary.
    min_freq filters rare keys (helps generalization).
    """
    from collections import Counter

    counter = Counter()
    for path in user_json_paths:
        with open(path, "r", encoding="utf-8") as handle:
            blob = json.load(handle)
            runs = blob.get("enrollment_runs", [])
            for run in runs:
                events = run.get("events", [])
                for ev in events:

```



```

        if ev.get("type") in ("keydown", "keyup"):
            counter[normalize_keysym(ev)] += 1

vocab = list(SPECIAL_TOKENS)
for token, freq in counter.most_common():
    if token in SPECIAL_TOKENS:
        continue
    if freq >= min_freq:
        vocab.append(token)

stoi = {tok: i for i, tok in enumerate(vocab)}
return stoi, vocab

```

./embedder.py

```

"""Compatibility entrypoint for the keystroke embedder training stack."""

from embedder_core.cli import main
from embedder_core.constants import MASK, PAD, SPECIAL_TOKENS, UNK
from embedder_core.data import AugmentConfig, KeystrokeUserDataset, collate_fn
from embedder_core.model import KeystrokeTransformer
from embedder_core.tokens import KeypressToken, parse_run_to_tokens
from embedder_core.train import TrainConfig, evaluate, load_model_weights, train
from embedder_core.utils import accuracy_from_logits, cosine_with_warmup, set_seed
from embedder_core.visualize import save_pca_embedding_plot
from embedder_core.vocab import build_keysym_vocab, normalize_keysym

__all__ = [
    "PAD",
    "UNK",
    "MASK",
    "SPECIAL_TOKENS",
    "KeypressToken",
    "AugmentConfig",
    "TrainConfig",
    "KeystrokeUserDataset",
    "KeystrokeTransformer",
    "accuracy_from_logits",
    "build_keysym_vocab",
    "collate_fn",
    "cosine_with_warmup",
    "evaluate",
    "load_model_weights",
    "main",
    "normalize_keysym",
    "parse_run_to_tokens",
    "save_pca_embedding_plot",
    "set_seed",
    "train",
]

if __name__ == "__main__":

```

```
raise SystemExit(main())
```

./jayadev.json

```
{
  "type": "keystroke_dataset",
  "version": 3,
  "created_unix": 1772319234,
  "run_format": "native_key_events_v1",
  "distance_threshold": 4.5,
  "feature_dim": 19,
  "num_enrollment_runs": 300,
  "num_test_runs": 0,
  "num_enrollment_raw_runs": 300,
  "num_test_raw_runs": 0,
  "enrollment_runs": [
    {
      "events": [
        {
          "type": "keydown",
          "timestamp_ms": 1772297956112.0,
          "keycode": 943782142,
          "keysym": "Shift_L",
          "char": ""
        },
        {
          "type": "keydown",
          "timestamp_ms": 1772297956305.0,
          "keycode": 289407060,
          "keysym": "T",
          "char": "T"
        },
        {
          "type": "keydown",
          "timestamp_ms": 1772297956411.0,
          "keycode": 67108968,
          "keysym": "h",
          "char": "h"
        },
        {
          "type": "keyup",
          "timestamp_ms": 1772297956528.0,
          "keycode": 67108968,
          "keysym": "h",
          "char": "h"
        },
        {
          "type": "keydown",
          "timestamp_ms": 1772297956529.0,
          "keycode": 97,
          "keysym": "a",
          "char": "a"
        }
      ]
    }
  ]
}
```

```
{
```

./jiahe.json

```
{
  "type": "keystroke_dataset",
  "version": 3,
  "created_unix": 1772317558,
  "run_format": "native_key_events_v1",
  "distance_threshold": 4.5,
  "feature_dim": 19,
  "num_enrollment_runs": 300,
  "num_test_runs": 0,
  "num_enrollment_raw_runs": 300,
  "num_test_raw_runs": 0,
  "enrollment_runs": [
    {
      "events": [
        {
          "type": "keydown",
          "timestamp_ms": 2047765.0,
          "keycode": 16,
          "keysym": "Shift_L",
          "char": ""
        },
        {
          "type": "keydown",
          "timestamp_ms": 2047843.0,
          "keycode": 65,
          "keysym": "A",
          "char": "A"
        },
        {
          "type": "keyup",
          "timestamp_ms": 2047921.0,
          "keycode": 16,
          "keysym": "Shift_L",
          "char": ""
        },
        {
          "type": "keydown",
          "timestamp_ms": 2047968.0,
          "keycode": 32,
          "keysym": "space",
          "char": " "
        },
        {
          "type": "keyup",
          "timestamp_ms": 2047968.0,
          "keycode": 65,
          "keysym": "a",
          "char": "a"
        }
      ]
    }
  ]
}
```

```
{
```

`./keystroke__app/init.py`

```
from .app import App, main
```

```
__all__ = ["App", "main"]
```

`./keystroke__app/app.py`

```
from copy import deepcopy
```

```
import tkinter as tk
```

```
from pathlib import Path
```

```
from tkinter import filedialog, messagebox, ttk
```

```
from typing import Any, Dict, List, Optional, Tuple
```

```
import numpy as np
```

```
from .capture import RunCapture, build_feature_vector_from_raw_run
```

```
from .config import (
    DATASET_VERSION,
    DEFAULT_DATASET_PATH,
    DEFAULT_DISTANCE_THRESHOLD,
    PROMPT_QUEUE_SIZE,
    SENTENCE_DATASET_PATH,
```

```
)
```

```
from .embedder_runtime import EmbedderRuntime, try_load_default_embedder_runtime
```

```
from .profile_manager import ProfileManager
```

```
from .prompts import PromptQueue, load_sentence_bank
```

```
from .storage import (
    SessionData,
    load_session_data,
    merge_session_data_file,
    save_session_data,
    set_feature_builder,
```

```
)
```

```
from .verifier import Verifier
```

```
SENTENCE_BANK = load_sentence_bank(SENTENCE_DATASET_PATH)
```

```
class App(tk.Tk):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.title("Keystroke Dynamics (Sentence Dataset) - Same vs Different")
```

```
        self.geometry("980x680")
```

```
        self.phase = "idle"  # idle / enroll / test / live_id / done
```

```
        self.phase_target_runs: Optional[int] = None
```

```
        self.prompt_queue = PromptQueue(SENTENCE_BANK, PROMPT_QUEUE_SIZE)
```

```
        self.capture = RunCapture()
```

```

self.verifier = Verifier()
self.profile_manager = ProfileManager()
self.embedder_runtime: Optional[EmbedderRuntime] = None
self.feature_backend_name: str = "handcrafted"
self._embedder_startup_message: str = ""

runtime, runtime_msg = try_load_default_embedder_runtime()
if runtime is not None:
    self.embedder_runtime = runtime
    self.feature_backend_name = "embedder"
    self._embedder_startup_message = runtime_msg
    set_feature_builder(runtime.build_feature_vector_from_raw_run)
else:
    self._embedder_startup_message = runtime_msg
    set_feature_builder(None)

self.enroll_samples: List[np.ndarray] = []
self.enroll_raw_runs: List[Dict[str, Any]] = []
self.test_samples: List[np.ndarray] = [] # cumulative, persisted with dataset save
self.test_raw_runs: List[Dict[str, Any]] = []
self.test_run_samples: List[np.ndarray] = [] # current test window only
self.test_run_raw_runs: List[Dict[str, Any]] = [] # current test window only

self.latest_phase_name: Optional[str] = None # "enroll" / "test"
self.latest_phase_data: Optional[SessionData] = None

self.profile_status_var = tk.StringVar(value="Loaded profiles: 0")

self.live_id_state = "unknown"
self.live_id_consecutive = 0
self.live_id_sentence_count = 0
self.live_id_votes: Dict[str, int] = {}
self.live_id_unknown_closest: Dict[str, int] = {}

self._build_ui()
self._render_prompt_queue()
self._set_status(
    f"Feature backend: {self.feature_backend_name}. Click 'Start Enrollment' or load a dataset."
)
if self._embedder_startup_message:
    self._log(self._embedder_startup_message)
self._update_progress_ui()
self._update_runs_label()
self._set_idle_controls()

def _build_ui(self):
    frm = ttk.Frame(self, padding=12)
    frm.pack(fill="both", expand=True)

    ttk.Label(
        frm,
        text="Type the top sentence exactly (including capitals and punctuation):",
        font=("Segoe UI", 12, "bold"),
    ).pack(anchor="w")

```

```

self.target_box = tk.Text(frm, height=10, wrap="word", font=("Consolas", 12))
self.target_box.pack(fill="x", pady=(6, 10))
self.target_box.configure(state="disabled")

ttk.Label(frm, text="Typing area:", font=("Segoe UI", 11, "bold")).pack(anchor="w")
self.input_box = tk.Text(frm, height=7, wrap="word", font=("Consolas", 12))
self.input_box.pack(fill="both", expand=False, pady=(6, 10))
self.input_box.tag_configure("mismatch", foreground="#c1121f")
self.input_box.focus_set()

self.input_box.bind("<KeyPress>", self._on_key_press)
self.input_box.bind("<KeyRelease>", self._on_key_release)

ctrl = ttk.Frame(frm)
ctrl.pack(fill="x", pady=(4, 10))

ttk.Label(ctrl, text="Enroll sentences:").pack(side="left")
self.enroll_target_var = tk.StringVar(value="20")
self.spn_enroll_target = ttk.Spinbox(
    ctrl,
    from_=1,
    to=500,
    increment=1,
    width=5,
    textvariable=self.enroll_target_var,
)
self.spn_enroll_target.pack(side="left", padx=(6, 10))

self.btn_enroll = ttk.Button(ctrl, text="Start Enrollment", command=self.start_enroll)
self.btn_enroll.pack(side="left")

ttk.Label(ctrl, text="Test sentences:").pack(side="left", padx=(12, 0))
self.test_target_var = tk.StringVar(value="10")
self.spn_test_target = ttk.Spinbox(
    ctrl,
    from_=1,
    to=500,
    increment=1,
    width=5,
    textvariable=self.test_target_var,
)
self.spn_test_target.pack(side="left", padx=(6, 10))

self.btn_test = ttk.Button(ctrl, text="Start Test", command=self.start_test, state="disabled")
self.btn_test.pack(side="left", padx=(8, 0))

self.btn_save_dataset = ttk.Button(
    ctrl,
    text="Save Dataset...",
    command=self.save_dataset,
    state="disabled",
)
self.btn_save_dataset.pack(side="left", padx=(8, 0))

```

```

self.btn_load_dataset = ttk.Button(
    ctrl,
    text="Load Dataset...",
    command=self.load_dataset,
)
self.btn_load_dataset.pack(side="left", padx=(8, 0))

self.btn_merge_dataset = ttk.Button(
    ctrl,
    text="Merge Dataset...",
    command=self.merge_dataset,
    state="disabled",
)
self.btn_merge_dataset.pack(side="left", padx=(8, 0))

self.btn_reset = ttk.Button(ctrl, text="Reset", command=self.reset_all)
self.btn_reset.pack(side="left", padx=(8, 0))

multi = ttk.LabelFrame(frm, text="Multi-Profile Identification", padding=8)
multi.pack(fill="both", expand=False, pady=(0, 10))

ttk.Label(multi, textvariable=self.profile_status_var).pack(anchor="w")

list_frame = ttk.Frame(multi)
list_frame.pack(fill="x", pady=(6, 6))

self.lst_profiles = tk.Listbox(list_frame, height=5, exportselection=False)
self.lst_profiles.pack(side="left", fill="both", expand=True)

scrollbar = ttk.Scrollbar(list_frame, orient="vertical", command=self.lst_profiles.yview)
scrollbar.pack(side="left", fill="y")
self.lst_profiles.configure(yscrollcommand=scrollbar.set)

multi_btns = ttk.Frame(multi)
multi_btns.pack(fill="x")

self.btn_load_profiles = ttk.Button(multi_btns, text="Load Profiles...", command=self.load_profiles)
self.btn_load_profiles.pack(side="left", padx=(0, 8))

self.btn_clear_profiles = ttk.Button(multi_btns, text="Clear Profiles", command=self.clear_profiles)
self.btn_clear_profiles.pack(side="left", padx=(0, 8))

cfg = ttk.Frame(multi)
cfg.pack(fill="x", pady=(4, 6))
ttk.Label(cfg, text="Sentences:").pack(side="left")
self.live_id_target_var = tk.StringVar(value="10")
self.spn_live_id_target = ttk.Spinbox(
    cfg,
    from_=0,
    to=500,
    increment=1,
    width=5,
    textvariable=self.live_id_target_var,

```

```

)
self.spn_live_id_target.pack(side="left", padx=(6, 6))
ttk.Label(cfg, text="(0 = unlimited)").pack(side="left")

live_btns = ttk.Frame(multi)
live_btns.pack(fill="x", pady=(4, 6))
self.btn_live_id_start = ttk.Button(live_btns, text="Start Live ID", command=self.start_live_id)
self.btn_live_id_start.pack(side="left", padx=(0, 8))
self.btn_live_id_stop = ttk.Button(live_btns, text="Stop Live ID", command=self.stop_live_id, s
self.btn_live_id_stop.pack(side="left")

self.lbl_live_id = ttk.Label(multi, text="Live: --", foreground="#555")
self.lbl_live_id.pack(anchor="w")
self.lbl_live_id_overall = ttk.Label(multi, text="Overall: --", foreground="#555")
self.lbl_live_id_overall.pack(anchor="w")

stats = ttk.Frame(frm)
stats.pack(fill="x", pady=(6, 0))

self.lbl_progress = ttk.Label(stats, text="Progress: --", font=("Segoe UI", 11, "bold"))
self.lbl_progress.pack(side="left")

self.lbl_runs = ttk.Label(stats, text="Runs: 0 enroll / 0 test", font=("Segoe UI", 11))
self.lbl_runs.pack(side="left", padx=(16, 0))

self.lbl_status = ttk.Label(frm, text="", font=("Segoe UI", 11))
self.lbl_status.pack(fill="x", pady=(10, 0))

self.output = tk.Text(frm, height=10, wrap="word", font=("Consolas", 11))
self.output.pack(fill="both", expand=True, pady=(10, 0))
self.output.configure(state="disabled")

def _log(self, msg: str):
    self.output.configure(state="normal")
    self.output.insert("end", msg + "\n")
    self.output.see("end")
    self.output.configure(state="disabled")

def _set_status(self, msg: str):
    self.lbl_status.configure(text=msg)

def _build_run_feature(self, raw_run: Dict[str, Any]) -> np.ndarray:
    if self.embedder_runtime is not None:
        return self.embedder_runtime.build_feature_vector_from_raw_run(raw_run)
    return build_feature_vector_from_raw_run(raw_run)

@staticmethod
def _first_mismatch_index(typed: str, target: str) -> Optional[int]:
    limit = min(len(typed), len(target))
    for i in range(limit):
        if typed[i] != target[i]:
            return i
    if len(typed) > len(target):

```



```

        return len(target)
    return None

def _update_input_mismatch_highlight(self, typed: str, target: str):
    self.input_box.tag_remove("mismatch", "1.0", "end")
    mismatch_idx = self._first_mismatch_index(typed, target)
    if mismatch_idx is None or mismatch_idx >= len(typed):
        return
    start = f"1.0+{mismatch_idx}c"
    end = f"1.0+{len(typed)}c"
    self.input_box.tag_add("mismatch", start, end)

def _update_runs_label(self):
    self.lbl_runs.configure(text=f"Runs: {len(self.enroll_samples)} enroll / {len(self.test_samples)}")

def _current_phase_run_count(self) -> int:
    if self.phase == "enroll":
        return len(self.enroll_samples)
    if self.phase == "test":
        return len(self.test_run_samples)
    if self.phase == "live_id":
        return self.live_id_sentence_count
    return 0

def _update_progress_ui(self):
    if self.phase in ("enroll", "test", "live_id") and self.phase_target_runs is not None:
        self.lbl_progress.configure(text=f"Progress: {self._current_phase_run_count()}/{self.phase_target_runs}")
    else:
        self.lbl_progress.configure(text="Progress: --")

def _update_profile_listbox(self):
    if not hasattr(self, "lst_profiles"):
        return
    names = self.profile_manager.list_profiles()
    self.lst_profiles.delete(0, "end")
    for name in names:
        self.lst_profiles.insert("end", name)
    self.profile_status_var.set(f"Loaded profiles: {len(names)}")
    self._update_live_id_button_state()

def _update_live_id_button_state(self):
    if not hasattr(self, "btn_live_id_start"):
        return
    can_start = self.phase in ("idle", "done") and len(self.profile_manager.list_profiles()) >= 1
    self.btn_live_id_start.configure(state="normal" if can_start else "disabled")
    self.btn_live_id_stop.configure(state="normal" if self.phase == "live_id" else "disabled")

    @staticmethod
    def _parse_target_runs(raw_value: str, label: str) -> int:
        try:
            target = int(raw_value)
        except Exception as ex:
            raise ValueError(f"{label} must be an integer.") from ex
        if target < 1:

```

```

        raise ValueError(f"{label} must be at least 1.")
    return target

def _has_session_data(self) -> bool:
    if self.latest_phase_data is None:
        return False
    return bool(self.latest_phase_data.enrollment_samples) or bool(self.latest_phase_data.test_samples)

def _clear_latest_phase_snapshot(self):
    self.latest_phase_name = None
    self.latest_phase_data = None

def _set_latest_phase_snapshot(self, phase_name: str):
    mean = None
    inv_cov = None
    if self.verifier.mean is not None:
        mean = np.array(self.verifier.mean, dtype=np.float32, copy=True)
    if self.verifier.inv_cov is not None:
        inv_cov = np.array(self.verifier.inv_cov, dtype=np.float32, copy=True)
    distance_threshold = float(self.verifier.distance_threshold)

    if phase_name == "enroll":
        data = SessionData(
            enrollment_samples=[np.array(s, dtype=np.float32, copy=True) for s in self.enrollment_samples],
            test_samples=[],
            enrollment_raw_runs=[deepcopy(run) for run in self.enrollment_raw_runs],
            test_raw_runs=[],
            enrollment_mean=mean,
            enrollment_inv_cov=inv_cov,
            distance_threshold=distance_threshold,
        )
    elif phase_name == "test":
        data = SessionData(
            enrollment_samples=[],
            test_samples=[np.array(s, dtype=np.float32, copy=True) for s in self.test_run_samples],
            enrollment_raw_runs=[],
            test_raw_runs=[deepcopy(run) for run in self.test_run_raw_runs],
            enrollment_mean=mean,
            enrollment_inv_cov=inv_cov,
            distance_threshold=distance_threshold,
        )
    else:
        raise ValueError(f"Unsupported phase snapshot: {phase_name}")

    self.latest_phase_name = phase_name
    self.latest_phase_data = data

def _set_running_controls(self):
    self.btn_enroll.configure(state="disabled")
    self.btn_test.configure(state="disabled")
    self.spn_enroll_target.configure(state="disabled")
    self.spn_test_target.configure(state="disabled")
    self.btn_save_dataset.configure(state="disabled")
    self.btn_load_dataset.configure(state="disabled")

```

```

self.btn_merge_dataset.configure(state="disabled")
self.btn_reset.configure(state="disabled")
self.btn_load_profiles.configure(state="disabled")
self.btn_clear_profiles.configure(state="disabled")
self.btn_live_id_start.configure(state="disabled")
self.btn_live_id_stop.configure(state="disabled")

def _set_idle_controls(self):
    self.btn_enroll.configure(state="normal")
    self.btn_test.configure(state="normal" if self.verifier.has_reference() else "disabled")
    self.spn_enroll_target.configure(state="normal")
    self.spn_test_target.configure(state="normal")
    has_data = self._has_session_data()
    self.btn_save_dataset.configure(state="normal" if has_data else "disabled")
    self.btn_load_dataset.configure(state="normal")
    self.btn_merge_dataset.configure(state="normal" if has_data else "disabled")
    self.btn_reset.configure(state="normal")
    self.btn_load_profiles.configure(state="normal")
    self.btn_clear_profiles.configure(state="normal")
    self._update_profile_listbox()
    self._update_live_id_button_state()

def _render_prompt_queue(self):
    self.target_box.configure(state="normal")
    self.target_box.delete("1.0", "end")
    self.target_box.insert("1.0", self.prompt_queue.as_text())
    self.target_box.configure(state="disabled")

def _reset_prompt_queue(self):
    self.prompt_queue.reset()
    self._render_prompt_queue()

def _clear_input(self):
    self.input_box.delete("1.0", "end")
    self.input_box.tag_remove("mismatch", "1.0", "end")
    self.capture.reset()

def load_profiles_dialog(self):
    paths = filedialog.askopenfilenames(
        title="Load Profile Dataset(s)",
        initialdir=str(DEFAULT_DATASET_PATH.parent),
        filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
    )
    if not paths:
        return

    loaded = []
    errors = []
    existing_names = set(self.profile_manager.list_profiles())

    for raw_path in paths:
        path = Path(raw_path)
        name = path.stem
        suffix = 2

```

```

        while name in existing_names:
            name = f"{path.stem}_{suffix}"
            suffix += 1
        try:
            self.profile_manager.load_profile(name, path)
            existing_names.add(name)
            loaded.append((name, path))
        except Exception as ex:
            errors.append(f"{path.name}: {ex}")

    self._update_profile_listbox()
    for name, path in loaded:
        self._log(f"Loaded profile '{name}' from {path.name}")
    if loaded:
        self._set_status(f"Loaded {len(loaded)} profile(s) for identification.")
    if errors:
        messagebox.showerror("Some profiles failed", "\n".join(errors))

def clear_profiles(self):
    self.profile_manager.clear()
    self._update_profile_listbox()
    self._log("Cleared all profiles.")
    self._update_live_id_button_state()

def _reset_live_id_state(self):
    self.live_id_state = "unknown"
    self.live_id_consecutive = 0
    self.live_id_sentence_count = 0
    self.live_id_votes.clear()
    self.live_id_unknown_closest.clear()
    self.lbl_live_id.configure(text="Live: --", foreground="#555")
    self.lbl_live_id_overall.configure(text="Overall: --", foreground="#555")

def start_live_id(self):
    if self.phase not in ("idle", "done"):
        return
    if len(self.profile_manager.list_profiles()) < 1:
        messagebox.showwarning("Need profiles", "Load at least one profile before Live ID.")
        return
    try:
        target = int(self.live_id_target_var.get())
    except Exception as ex:
        messagebox.showwarning("Invalid setting", f"Sentences must be an integer.\n{ex}")
        return
    if target < 0:
        messagebox.showwarning("Invalid setting", "Sentences must be >= 0.")
        return

    self.phase = "live_id"
    self.phase_target_runs = target if target > 0 else None
    self._reset_live_id_state()
    self._reset_prompt_queue()
    self._set_running_controls()
    self.btn_live_id_stop.configure(state="normal")

```

```

self._update_progress_ui()
self._set_status("LIVE ID: Type sentences to identify typist. Click 'Stop Live ID' when done.")
profiles = ", ".join(self.profile_manager.list_profiles())
self._log(f"\n[Live ID started] profiles: {profiles if profiles else 'none'}")
self._clear_input()
self._update_live_id_button_state()

def stop_live_id(self):
    if self.phase != "live_id":
        return
    self._finish_live_id(stopped=True)

def _finish_live_id(self, stopped: bool):
    if self.phase != "live_id":
        return
    total = self.live_id_sentence_count
    self.phase = "idle"
    self.phase_target_runs = None
    overall_text, overall_color = self._compute_live_id_overall()
    self.lbl_live_id_overall.configure(text=overall_text, foreground=overall_color)
    reason = "stopped" if stopped else "finished"
    self._log(f"[Live ID {reason}] {total} sentence(s) typed")
    if total > 0:
        summary = overall_text.replace("Overall: ", "")
        self._log(f"Final verdict: {summary}")
        self._set_status(f"Live ID result: {summary}")
    else:
        self._set_status("Live ID session ended (no sentences).")
    self._render_prompt_queue()
    self._clear_input()
    self._update_progress_ui()
    self._set_idle_controls()

def _compute_live_id_overall(self) -> Tuple[str, str]:
    total = self.live_id_sentence_count
    if total == 0:
        return "Overall: --", "#555"

    total_identified = sum(self.live_id_votes.values())
    total_unknown = sum(self.live_id_unknown_closest.values())

    if total_identified > total_unknown and self.live_id_votes:
        overall_name = max(self.live_id_votes, key=self.live_id_votes.get)
        overall_count = self.live_id_votes[overall_name]
        return (
            f"Overall: IDENTIFIED: {overall_name} ({overall_count}/{total} sentences)",
            "#1b9c85",
        )

    if self.live_id_unknown_closest:
        overall_closest = max(self.live_id_unknown_closest, key=self.live_id_unknown_closest.get)
    else:
        overall_closest = "n/a"
    overall_count = total_unknown

```

```

        return (
            f"Overall: UNKNOWN - closest: {overall_closest} ({overall_count}/{total} sentences)",
            "#c1121f",
        )

def _live_id_score(self, feat: np.ndarray):
    expected_dim = self.profile_manager.expected_feature_dim()
    if expected_dim is not None and feat.shape[0] != expected_dim:
        self._log("Rejected LIVE ID run: feature dimension mismatch with loaded profiles.")
        return

    X = feat.reshape(1, -1)
    try:
        best_name, best_distance, distances = self.profile_manager.identify(X)
    except Exception as ex:
        self._log(f"Live ID identify failed: {ex}")
        self._finish_live_id(stopped=True)
        return

    if not distances:
        return

    closest_name = min(distances, key=distances.get)
    closest_dist = float(distances[closest_name])

    self.live_id_sentence_count += 1
    sentence_idx = self.live_id_sentence_count

    if best_name is not None:
        self.live_id_votes[best_name] = self.live_id_votes.get(best_name, 0) + 1
    else:
        self.live_id_unknown_closest[closest_name] = self.live_id_unknown_closest.get(closest_name, 0) + 1

    new_state = "identified" if best_name is not None else "unknown"
    if new_state == self.live_id_state:
        self.live_id_consecutive += 1
    else:
        self.live_id_state = new_state
        self.live_id_consecutive = 1

    if self.live_id_consecutive >= 3 or sentence_idx == 1:
        display_dist = float(best_distance) if best_name is not None else closest_dist
        if self.live_id_state == "identified":
            label_text = f"Live: IDENTIFIED: {best_name} (dist: {display_dist:.2f})"
            label_color = "#1b9c85"
        else:
            label_text = f"Live: UNKNOWN - closest: {closest_name} (dist: {closest_dist:.2f})"
            label_color = "#c1121f"
        self.lbl_live_id.configure(text=label_text, foreground=label_color)

    overall_text, overall_color = self._compute_live_id_overall()
    self.lbl_live_id_overall.configure(text=overall_text, foreground=overall_color)

    all_dists_str = " | ".join(f"{k}: {v:.2f}" for k, v in sorted(distances.items(), key=lambda x: x[1]))

```

```

verdict = f"IDENTIFIED: {best_name}" if best_name is not None else f"UNKNOWN (closest: {closest})"
self._log(f"[sentence {sentence_idx}] {all_dists_str} → {verdict}")

if self.phase_target_runs is not None and sentence_idx >= self.phase_target_runs:
    self._finish_live_id(stopped=False)

def _apply_session_data(self, data: SessionData):
    self.enroll_samples = [np.array(s, dtype=np.float32, copy=True) for s in data.enrollment_samples]
    self.test_samples = [np.array(s, dtype=np.float32, copy=True) for s in data.test_samples]
    self.enroll_raw_runs = [deepcopy(run) for run in data.enrollment_raw_runs]
    self.test_raw_runs = [deepcopy(run) for run in data.test_raw_runs]
    self.test_run_samples.clear()
    self.test_run_raw_runs.clear()

    self.verifier.clear()
    self.verifier.distance_threshold = float(DEFAULT_DISTANCE_THRESHOLD)

    if len(self.enroll_samples) >= 3:
        try:
            X = np.stack(self.enroll_samples, axis=0)
            self.verifier.fit(X)
        except ValueError as ex:
            self._log(f"Verifier fit failed while loading dataset: {ex}")
    elif data.enrollment_mean is not None and data.enrollment_inv_cov is not None:
        # Fall back to stored stats if available (e.g., test-only snapshot).
        self.verifier.mean = np.array(data.enrollment_mean, dtype=np.float32, copy=True)
        self.verifier.inv_cov = np.array(data.enrollment_inv_cov, dtype=np.float32, copy=True)
        self.verifier._n_enrollment_runs = len(self.enroll_samples)

    self._update_runs_label()
    self._update_progress_ui()
    self._set_idle_controls()

def reset_all(self):
    self.phase = "idle"
    self.phase_target_runs = None
    self.enroll_samples.clear()
    self.enroll_raw_runs.clear()
    self.test_samples.clear()
    self.test_raw_runs.clear()
    self.test_run_samples.clear()
    self.test_run_raw_runs.clear()
    self._clear_latest_phase_snapshot()
    self.capture.reset()
    self.verifier.clear()
    self.verifier.distance_threshold = float(DEFAULT_DISTANCE_THRESHOLD)
    self._reset_prompt_queue()
    self._clear_input()
    self._update_runs_label()
    self._update_progress_ui()
    self._set_idle_controls()
    self._set_status("Reset. Click 'Start Enrollment' or load a dataset.")
    self._log("\n[Reset]\n")

```

```

def load_dataset(self):
    if self.phase in ("enroll", "test", "live_id"):
        messagebox.showwarning("Busy", "Stop the current run before loading a dataset.")
        return

    path_str = filedialog.askopenfilename(
        title="Load Dataset",
        initialdir=str(DEFAULT_DATASET_PATH.parent),
        filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
    )
    if not path_str:
        return

    path = Path(path_str)
    try:
        data = load_session_data(path)
    except Exception as ex:
        messagebox.showerror("Load failed", f"Could not load dataset:\n{ex}")
        return

    self._clear_latest_phase_snapshot()
    self._apply_session_data(data)
    self._reset_prompt_queue()
    self._clear_input()
    self._log(f"Loaded dataset from: {path}")
    self._set_status(f"Dataset loaded: {path.name}. Save/Merge now use next completed phase only.")

def save_dataset(self):
    if not self._has_session_data():
        messagebox.showwarning("No data", "No recent phase data is available to save. Complete enroll")
        return

    path_str = filedialog.asksaveasfilename(
        title="Save Dataset",
        defaultextension=".json",
        initialdir=str(DEFAULT_DATASET_PATH.parent),
        initialfile=DEFAULT_DATASET_PATH.name,
        filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
    )
    if not path_str:
        return

    path = Path(path_str)
    try:
        assert self.latest_phase_data is not None
        save_session_data(path, self.latest_phase_data, DATASET_VERSION)
    except Exception as ex:
        messagebox.showerror("Save failed", f"Could not save dataset:\n{ex}")
        return

    phase_label = self.latest_phase_name if self.latest_phase_name is not None else "phase"
    self._log(f"Saved latest {phase_label} dataset to: {path}")
    self._set_status(f"Dataset saved: {path.name}")
    self._set_idle_controls()

```



```

def merge_dataset(self):
    if not self._has_session_data():
        messagebox.showwarning("No data", "No recent phase data is available to merge. Complete enr
        return

    path_str = filedialog.askopenfilename(
        title="Select Dataset JSON to Merge Into",
        initialdir=str(DEFAULT_DATASET_PATH.parent),
        filetypes=[("JSON files", "*.json"), ("All files", "*.*")],
    )
    if not path_str:
        return

    path = Path(path_str)
    try:
        assert self.latest_phase_data is not None
        merged = merge_session_data_file(path, self.latest_phase_data, DATASET_VERSION)
        self._apply_session_data(merged)
    except Exception as ex:
        messagebox.showerror("Merge failed", f"Could not merge dataset:\n{ex}")
        return

    phase_label = self.latest_phase_name if self.latest_phase_name is not None else "phase"
    self._log(f"Merged latest {phase_label} dataset into: {path}")
    self._set_status(f"Merged into dataset: {path.name}")

def start_enroll(self):
    try:
        target_runs = self._parse_target_runs(self.enroll_target_var.get(), "Enroll sentences")
    except Exception as ex:
        messagebox.showwarning("Invalid setting", str(ex))
        return

    self.reset_all()
    self.phase = "enroll"
    self.phase_target_runs = target_runs
    self._reset_prompt_queue()
    self._set_running_controls()
    self._update_progress_ui()
    self._set_status(f"ENROLLMENT: Type {target_runs} sentence(s) exactly.")
    self._log(f"[Enrollment started] target runs = {target_runs}")

def start_test(self):
    if self.phase not in ("idle", "done"):
        return
    if not self.verifier.has_reference():
        messagebox.showwarning("Not enough enrollment", "Enroll first or load a dataset with enroll

    try:
        target_runs = self._parse_target_runs(self.test_target_var.get(), "Test sentences")
    except Exception as ex:
        messagebox.showwarning("Invalid setting", str(ex))

```

```

        return

    self.phase = "test"
    self.phase_target_runs = target_runs
    self.test_run_samples.clear()
    self.test_run_raw_runs.clear()
    self._reset_prompt_queue()
    self._set_running_controls()
    self._update_progress_ui()
    self._set_status(f"TEST: Type {target_runs} sentence(s) exactly; completed lines will rotate.")
    self._log(f"\n[Test started] target runs = {target_runs}")
    self._clear_input()

def _finish_phase(self):
    if self.phase == "enroll":
        accepted = len(self.enroll_samples)
        self._log(f"[Enrollment finished] accepted runs = {accepted}")
        if accepted < 3:
            self._log("Need at least 3 enrollment runs to build a profile. Try again.")
            self._set_status("Enrollment needs at least 3 completed runs. Click 'Start Enrollment' ")
            self.phase = "idle"
            self.phase_target_runs = None
            self._set_idle_controls()
            self._update_progress_ui()
            return

        X = np.stack(self.enroll_samples, axis=0)
        self.verifier.fit(X)
        self._set_latest_phase_snapshot("enroll")
        self._log(f"Enrollment profile built from {accepted} run(s).")
        self._set_status(f"Enrollment complete. Built profile from {accepted} runs.")
        self.phase = "idle"
        self._clear_input()
        self._set_idle_controls()

    elif self.phase == "test":
        self._log(f"[Test finished] accepted runs = {len(self.test_run_samples)}")
        if len(self.test_run_samples) < 1:
            self._log("No test runs captured. Try again and complete the prompt.")
            self._set_status("No test runs captured. Click 'Start Test' again and type continuously")
            self.phase = "idle"
            self.phase_target_runs = None
            self._set_idle_controls()
            self._update_progress_ui()
            return

        Xtest = np.stack(self.test_run_samples, axis=0)
        try:
            distances, inlier = self.verifier.score(Xtest)
        except Exception as ex:
            self._log(f"Scoring failed: {ex}")
            self._set_status("Scoring failed due to reference/data mismatch. Re-enroll or load another")
            self.phase = "idle"
            self.phase_target_runs = None

```

```

        self._set_idle_controls()
        self._update_progress_ui()
        return

    inlier_frac = float(inlier.mean())
    avg_distance = float(distances.mean())
    min_distance = float(distances.min())
    max_distance = float(distances.max())

    self._log("\n--- RESULT ---")
    self._log(f"Avg Mahalanobis distance: {avg_distance:.4f} (lower = more like enrolled user)")
    self._log(
        f"Distance range: [{min_distance:.4f}, {max_distance:.4f}] "
        f"vs threshold {self.verifier.distance_threshold:.2f}"
    )
    self._log(f"Inlier fraction: {inlier_frac:.2%} (runs classified as 'User A-like')")

    same = inlier_frac >= 0.60
    self._log("VERDICT: " + ("SAME PERSON (likely)" if same else "DIFFERENT PERSON (likely)"))

    self._set_latest_phase_snapshot("test")
    self._set_status("Test done. Save/Merge now use this test snapshot.")
    self.phase = "idle"
    self._set_idle_controls()

    self.phase_target_runs = None
    self._update_progress_ui()
    self._update_runs_label()

def _maybe_accept_run(self):
    if self.phase not in ("enroll", "test", "live_id"):
        return
    typed = self.input_box.get("1.0", "end-1c")
    target = self.prompt_queue.current()
    self._update_input_mismatch_highlight(typed, target)

    if typed == target:
        raw_run = self.capture.build_raw_run()
        if raw_run is None:
            self._log("Run matched text but no usable timing data was captured; avoid paste and type")
        else:
            feat = self._build_run_feature(raw_run)
            if self.phase == "enroll":
                self.enroll_samples.append(feat)
                self.enroll_raw_runs.append(deepcopy(raw_run))
                self._log(f"Accepted ENROLL run #{len(self.enroll_samples)}")
            elif self.phase == "test":
                if self.verifier.mean is not None and feat.shape[0] != self.verifier.mean.shape[0]:
                    self._log("Rejected TEST run: feature dimension mismatch with loaded reference.")
                else:
                    self.test_run_samples.append(feat)
                    self.test_samples.append(feat)
                    self.test_run_raw_runs.append(deepcopy(raw_run))
                    self.test_raw_runs.append(deepcopy(raw_run))

```

```

        self._log(
            f"Accepted TEST run #{len(self.test_run_samples)} "
            f"(total saved tests: {len(self.test_samples)})"
        )
    elif self.phase == "live_id":
        self._live_id_score(feet)

    self._update_runs_label()
    self._update_progress_ui()

    # Consume top line and append a new sentence at the bottom.
    self.prompt_queue.advance()
    self._render_prompt_queue()
    self._clear_input()
    if self.phase_target_runs is not None and self._current_phase_run_count() >= self.phase_target_runs:
        self._finish_phase()
    return

if len(typed) > len(target) + 5:
    self._set_status("You overshoot the prompt. Use Reset or backspace; aim to match exactly.")

def _on_key_press(self, event):
    if self.phase not in ("enroll", "test", "live_id"):
        return

    # Capture all keydown events (including modifiers) for raw dataset logging.
    self.capture.on_key_press(event)

    if event.keysym in {"Shift_L", "Shift_R", "Control_L", "Control_R", "Alt_L", "Alt_R", "Caps_Lock"}:
        return

    if event.keysym == "BackSpace":
        # Allow corrections while still re-evaluating the run afterward.
        self.after(1, self._maybe_accept_run)
        return

    if event.keysym in {"Delete", "Left", "Right", "Up", "Down", "Home", "End", "Return"}:
        return "break"

    ch = event.char
    if ch == "":
        return "break"

    # Force append-at-end behavior even if the caret was moved manually.
    self.input_box.tag_remove("sel", "1.0", "end")
    self.input_box.mark_set("insert", "end-1c")
    self.after(1, self._maybe_accept_run)

def _on_key_release(self, event):
    if self.phase not in ("enroll", "test", "live_id"):
        return
    self.capture.on_key_release(event)

```

```
def main():
    app = App()
    app.mainloop()
```

./keystroke__app/capture.py

```
import time
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

import numpy as np

RAW_RUN_FORMAT = "native_key_events_v1"

@dataclass
class KeyEventRec:
    kind: str
    timestamp_ms: float
    keycode: int
    keysym: str
    char: str

class RunCapture:
    """
    Captures native keydown/keyup keyboard events for one run.
    We capture all keydown/keyup events; text features are derived from printable keydowns.
    """

    def __init__(self):
        self.events: List[KeyEventRec] = []
        self.active_down_counts: Dict[int, int] = {}

    def reset(self):
        self.events.clear()
        self.active_down_counts.clear()

    @staticmethod
    def _event_timestamp_ms(event) -> float:
        native_ts = getattr(event, "time", None)
        if isinstance(native_ts, (int, float)):
            return float(native_ts)
        return float(time.time() * 1000.0)

    def on_key_press(self, event):
        ch = str(getattr(event, "char", ""))
        keycode = int(event.keycode)
        self.events.append(
            KeyEventRec(
                kind="keydown",
                timestamp_ms=self._event_timestamp_ms(event),
                keycode=keycode,
```

```

        keysym=str(getattr(event, "keysym", "")),
        char=ch,
    )
)
self.active_down_counts[keycode] = int(self.active_down_counts.get(keycode, 0) + 1)

def on_key_release(self, event):
    keycode = int(event.keycode)
    if int(self.active_down_counts.get(keycode, 0)) <= 0:
        return
    self.events.append(
        KeyEventRec(
            kind="keyup",
            timestamp_ms=self._event_timestamp_ms(event),
            keycode=keycode,
            keysym=str(getattr(event, "keysym", "")),
            char=str(getattr(event, "char", "")),
        )
    )
    remaining = int(self.active_down_counts.get(keycode, 0)) - 1
    if remaining > 0:
        self.active_down_counts[keycode] = remaining
    else:
        self.active_down_counts.pop(keycode, None)

@staticmethod
def _timing_stats(values: np.ndarray) -> np.ndarray:
    if values.size == 0:
        return np.zeros(6, dtype=np.float32)
    q10, q50, q90 = np.quantile(values, [0.10, 0.50, 0.90])
    return np.array(
        [
            float(values.mean()),
            float(values.std()),
            float(q10),
            float(q50),
            float(q90),
            float(np.max(values)),
        ],
        dtype=np.float32,
    )

@classmethod
def _collect_run_stream_from_events(cls, events: List[Dict[str, Any]]) -> Optional[Tuple[List[str],
down_stack: Dict[int, List[int]] = {}
chars_raw: List[str] = []
down_times_s: List[float] = []
dwell_raw: List[Optional[float]] = []

for ev in events:
    kind = str(ev["type"])
    keycode = int(ev["keycode"])
    t_s = float(ev["timestamp_ms"]) / 1000.0

```

```

if kind == "keydown":
    ch = str(ev.get("char", ""))
    is_text_key = ch == " " or (ch != "" and ch.isprintable())
    if not is_text_key:
        continue

    chars_raw.append(ch)
    down_times_s.append(t_s)
    dwell_raw.append(None)
    idx = len(chars_raw) - 1
    down_stack.setdefault(keycode, []).append(idx)
    continue

if kind == "keyup":
    stack = down_stack.get(keycode)
    if not stack:
        continue
    idx = stack.pop()
    dwell = t_s - down_times_s[idx]
    if 0.005 <= dwell <= 2.0:
        dwell_raw[idx] = dwell

dwell_list: List[float] = []
flight_list: List[float] = []
chars: List[str] = []

if not down_times_s:
    return None

prev_down: Optional[float] = None
flights_raw: List[float] = []
for t_down in down_times_s:
    if prev_down is None:
        flights_raw.append(0.0)
    else:
        flights_raw.append(max(0.0, t_down - prev_down))
    prev_down = t_down

last_was_space = False
for i, ch in enumerate(chars_raw):
    if ch.isspace():
        if last_was_space:
            continue
        ch = " "
        last_was_space = True
    else:
        last_was_space = False

d = dwell_raw[i] if i < len(dwell_raw) and dwell_raw[i] is not None else 0.0
f = flights_raw[i] if i < len(flights_raw) else 0.0
dwell_list.append(d)
flight_list.append(f)
chars.append(ch)

```

```

    if not dwell_list:
        return None

    dwell = np.array(dwell_list, dtype=np.float32)
    flight = np.array(flight_list, dtype=np.float32)

    dwell = np.clip(dwell, 0.0, 2.0)
    flight = np.clip(flight, 0.0, 2.0)
    return chars, dwell, flight

def _collect_run_stream(self) -> Optional[Tuple[List[str], np.ndarray, np.ndarray]]:
    raw_run = self.build_raw_run()
    if raw_run is None:
        return None
    return self._collect_run_stream_from_events(raw_run["events"])

@classmethod
def _feature_vector_from_parts(
    cls,
    chars: List[str],
    dwell: np.ndarray,
    flight: np.ndarray,
) -> np.ndarray:
    dwell_stats = cls._timing_stats(dwell)
    flight_stats = cls._timing_stats(flight)
    total_chars = float(len(chars))
    uppercase_ratio = float(sum(c.isalpha() and c.isupper() for c in chars) / max(1.0, total_chars))
    punctuation_ratio = float(sum(c in ".,:!?" for c in chars) / max(1.0, total_chars))
    space_ratio = float(sum(c == " " for c in chars) / max(1.0, total_chars))
    long_pause_ratio = float((flight > 0.35).mean()) if flight.size else 0.0
    total_dwell = float(dwell.sum())
    total_flight = float(flight.sum())

    feat = np.concatenate(
        [
            np.log1p(dwell_stats),
            np.log1p(flight_stats),
            np.array(
                [
                    np.log1p(total_chars),
                    uppercase_ratio,
                    punctuation_ratio,
                    space_ratio,
                    long_pause_ratio,
                    np.log1p(total_dwell),
                    np.log1p(total_flight),
                ]
            ),
        ],
        dtype=np.float32,
    ),
    axis=0,
).astype(np.float32)
return feat

```



```

def build_raw_run(self) -> Optional[Dict[str, Any]]:
    if not self.events:
        return None

    events = [
        {
            "type": ev.kind,
            "timestamp_ms": float(ev.timestamp_ms),
            "keycode": int(ev.keycode),
            "keysym": str(ev.keysym),
            "char": str(ev.char),
        }
        for ev in self.events
    ]
    return {
        "events": events,
    }

def build_feature_vector(self) -> Optional[np.ndarray]:
    stream = self._collect_run_stream()
    if stream is None:
        return None
    chars, dwell, flight = stream
    return self._feature_vector_from_parts(chars, dwell, flight)

def normalize_raw_run(raw_run: Dict[str, Any]) -> Dict[str, Any]:
    if not isinstance(raw_run, dict):
        raise ValueError("Raw run must be an object.")

    events = raw_run.get("events")
    if not isinstance(events, list):
        raise ValueError("Raw run must include an 'events' list.")
    if not events:
        raise ValueError("Raw run must include at least one keyboard event.")

    out_events: List[Dict[str, Any]] = []
    last_ts: Optional[float] = None

    for i, event in enumerate(events):
        if not isinstance(event, dict):
            raise ValueError(f"events[{i}] must be an object.")

        kind = str(event.get("type", ""))
        if kind not in {"keydown", "keyup"}:
            raise ValueError(f"events[{i}].type must be 'keydown' or 'keyup'.")

        timestamp_ms_raw = event.get("timestamp_ms")
        if not isinstance(timestamp_ms_raw, (int, float)):
            raise ValueError(f"events[{i}].timestamp_ms must be numeric.")
        timestamp_ms = float(timestamp_ms_raw)
        if timestamp_ms < 0.0:
            raise ValueError(f"events[{i}].timestamp_ms must be >= 0.")
        if last_ts is not None and timestamp_ms < last_ts:

```

```

        raise ValueError("Event timestamps must be non-decreasing within a run.")
    last_ts = timestamp_ms

    keycode_raw = event.get("keycode")
    if not isinstance(keycode_raw, (int, float)):
        raise ValueError(f"events[{i}].keycode must be numeric.")
    keycode = int(keycode_raw)

    keysym = str(event.get("keysym", ""))
    char = str(event.get("char", ""))

    out_events.append(
        {
            "type": kind,
            "timestamp_ms": timestamp_ms,
            "keycode": keycode,
            "keysym": keysym,
            "char": char,
        }
    )

    if not any(ev["type"] == "keydown" for ev in out_events):
        raise ValueError("Raw run must include at least one keydown event.")

    return {
        "events": out_events,
    }

def build_feature_vector_from_raw_run(raw_run: Dict[str, Any]) -> np.ndarray:
    normalized = normalize_raw_run(raw_run)
    stream = RunCapture._collect_run_stream_from_events(normalized["events"])
    if stream is None:
        raise ValueError("Raw run has no usable keydown timing data.")
    chars, dwell, flight = stream
    return RunCapture._feature_vector_from_parts(chars, dwell, flight)

```

./keystroke__app/config.py

```

from pathlib import Path

APP_ROOT = Path(__file__).resolve().parent.parent

PROMPT_QUEUE_SIZE = 5
SENTENCE_DATASET_PATH = APP_ROOT / "medium_english_sentences.txt"

DEFAULT_DATASET_PATH = APP_ROOT / "keystroke_dataset.json"
DEFAULT_EMBEDDER_FINAL_WEIGHTS_PATH = APP_ROOT / "keystroke_user_classifier_keysym_final_weights.pt"
DEFAULT_EMBEDDER_CHECKPOINT_PATH = APP_ROOT / "keystroke_user_classifier_keysym.pt"

DATASET_VERSION = 3

DEFAULT_DISTANCE_THRESHOLD = 4.5

```

```
LEGACY_SCORE_THRESHOLD = 0.72
```

./keystroke__app/embedder__runtime.py

```
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Dict, Optional, Tuple

import numpy as np

from .capture import normalize_raw_run
from .config import DEFAULT_EMBEDDER_CHECKPOINT_PATH, DEFAULT_EMBEDDER_FINAL_WEIGHTS_PATH

@dataclass
class EmbedderRuntime:
    model: Any
    stoi: Dict[str, int]
    device: str
    max_tokens: int
    clip_ms: float
    log1p: bool
    weights_path: Path
    checkpoint_path: Optional[Path]

    @property
    def feature_dim(self) -> int:
        return int(self.model.d_model)

    def build_feature_vector_from_raw_run(self, raw_run: Dict[str, Any]) -> np.ndarray:
        import torch
        from embedder_core.tokens import parse_run_to_tokens

        normalized = normalize_raw_run(raw_run)
        tokens = parse_run_to_tokens(normalized["events"], self.stoi)
        if not tokens:
            raise ValueError("Raw run has no usable keypress tokens for embedder inference.")

        if len(tokens) > self.max_tokens:
            tokens = tokens[: self.max_tokens]

        keySYM_ids = torch.tensor([[t.keySYM_id for t in tokens]], dtype=torch.long, device=self.device)
        dwell = torch.tensor([[t.dwell_ms for t in tokens]], dtype=torch.float32, device=self.device)
        flight = torch.tensor([[t.flight_ms for t in tokens]], dtype=torch.float32, device=self.device)
        attn_mask = torch.ones((1, keySYM_ids.shape[1]), dtype=torch.bool, device=self.device)

        dwell = torch.clamp(dwell, 0.0, float(self.clip_ms))
        flight = torch.clamp(flight, 0.0, float(self.clip_ms))
        if self.log1p:
            dwell = torch.log1p(dwell)
            flight = torch.log1p(flight)

        with torch.no_grad():
```

```

        emb = self.model.encode(keysym_ids, dwell, flight, attn_mask)
    return emb.detach().cpu().numpy().reshape(-1).astype(np.float32)

def _extract_model_state(payload: Any) -> Dict[str, Any]:
    if isinstance(payload, dict) and isinstance(payload.get("model_state"), dict):
        return payload["model_state"]
    if isinstance(payload, dict):
        if payload and all(hasattr(v, "shape") for v in payload.values()):
            return payload
    raise ValueError("Invalid embedder weights file format.")

def _infer_arch(state: Dict[str, Any], train_cfg: Dict[str, Any]) -> Dict[str, Any]:
    key_emb = state.get("key_emb.weight")
    head_w = state.get("head.1.weight")
    if key_emb is None or head_w is None:
        raise ValueError("Weights are missing required tensors (key_emb/head).")

    vocab_size = int(key_emb.shape[0])
    d_model = int(key_emb.shape[1])
    num_classes = int(head_w.shape[0])

    layer_ids = set()
    for name in state.keys():
        if name.startswith("encoder.layers."):
            parts = name.split(".")
            if len(parts) > 2 and parts[2].isdigit():
                layer_ids.add(int(parts[2]))
    num_layers = max(layer_ids) + 1 if layer_ids else int(train_cfg.get("num_layers", 3))

    ff_w = state.get("encoder.layers.0.linear1.weight")
    dim_ff = int(ff_w.shape[0]) if ff_w is not None else int(train_cfg.get("dim_ff", max(2 * d_model, 1)))

    nhead = int(train_cfg.get("nhead", 4))
    dropout = float(train_cfg.get("dropout", 0.1))
    use_cls_token = "cls" in state

    return {
        "num_classes": num_classes,
        "vocab_size": vocab_size,
        "d_model": d_model,
        "nhead": nhead,
        "num_layers": num_layers,
        "dim_ff": dim_ff,
        "dropout": dropout,
        "use_cls_token": use_cls_token,
    }

def _load_torch_payload(path: Path) -> Any:
    import torch

    return torch.load(str(path), map_location="cpu")

```

```

def load_embedder_runtime(
    weights_path: Path,
    checkpoint_path: Optional[Path] = None,
) -> EmbedderRuntime:
    import torch
    from embedder_core.model import KeystrokeTransformer

    if not weights_path.exists():
        raise FileNotFoundError(f"Embedder weights file not found: {weights_path}")

    weights_payload = _load_torch_payload(weights_path)
    model_state = _extract_model_state(weights_payload)

    meta_payload = weights_payload if isinstance(weights_payload, dict) else {}
    stoi = meta_payload.get("stoi") if isinstance(meta_payload, dict) else None
    train_cfg = meta_payload.get("train_cfg", {}) if isinstance(meta_payload, dict) else {}
    aug_cfg = meta_payload.get("aug_cfg", {}) if isinstance(meta_payload, dict) else {}

    ckpt_used = None
    if not isinstance(stoi, dict):
        if checkpoint_path is None:
            raise ValueError("Embedder weights file has no vocab metadata and no checkpoint path was provided")
        if not checkpoint_path.exists():
            raise FileNotFoundError(
                f"Embedder weights file lacks vocab metadata and checkpoint was not found: {checkpoint_path}"
            )
        ckpt_payload = _load_torch_payload(checkpoint_path)
        if not isinstance(ckpt_payload, dict) or not isinstance(ckpt_payload.get("stoi"), dict):
            raise ValueError("Checkpoint file must include 'stoi' to run embedder inference.")
        stoi = ckpt_payload["stoi"]
        if not train_cfg:
            train_cfg = ckpt_payload.get("train_cfg", {})
        if not aug_cfg:
            aug_cfg = ckpt_payload.get("aug_cfg", {})
        ckpt_used = checkpoint_path

    arch = _infer_arch(model_state, train_cfg if isinstance(train_cfg, dict) else {})
    model = KeystrokeTransformer(**arch)
    model.load_state_dict(model_state, strict=True)
    model.eval()

    device = "cpu"
    model = model.to(device)

    max_tokens = int((train_cfg or {}).get("max_tokens", 256))
    clip_ms = float((aug_cfg or {}).get("clip_ms", 800.0))
    log1p = bool((aug_cfg or {}).get("log1p", True))

    return EmbedderRuntime(
        model=model,
        stoi=stoi,
        device=device,

```

```

        max_tokens=max_tokens,
        clip_ms=clip_ms,
        loglp=loglp,
        weights_path=weights_path,
        checkpoint_path=ckpt_used,
    )

def try_load_default_embedder_runtime() -> Tuple[Optional[EmbedderRuntime], str]:
    try:
        runtime = load_embedder_runtime(
            weights_path=DEFAULT_EMBEDDER_FINAL_WEIGHTS_PATH,
            checkpoint_path=DEFAULT_EMBEDDER_CHECKPOINT_PATH,
        )
    except Exception as ex:
        return None, f"Embedder disabled: {ex}"

    if runtime.checkpoint_path is None:
        msg = f"Embedder enabled from {runtime.weights_path.name}"
    else:
        msg = (
            f"Embedder enabled from {runtime.weights_path.name} "
            f"(vocab metadata from {runtime.checkpoint_path.name})"
        )
    return runtime, msg

```

./keystroke__app/profile__manager.py

```

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Optional, Tuple

import numpy as np

from .storage import load_session_data
from .verifier import Verifier

@dataclass
class ProfileEntry:
    name: str
    dataset_path: Path
    verifier: Verifier

class ProfileManager:
    def __init__(self):
        self._profiles: Dict[str, ProfileEntry] = {}

    def load_profile(self, profile_name: str, path: Path):
        data = load_session_data(path)

```

```

    if not data.enrollment_samples:
        raise ValueError("Dataset has no enrollment runs to build a profile.")

    verifier = Verifier()
    X = np.stack(data.enrollment_samples, axis=0)
    verifier.fit(X)
    if data.distance_threshold:
        verifier.distance_threshold = float(data.distance_threshold)

    self._profiles[profile_name] = ProfileEntry(name=profile_name, dataset_path=path, verifier=verifier)

def identify(
    self, X: np.ndarray, unknown_threshold: float = 4.5
) -> Tuple[Optional[str], float, Dict[str, float]]:
    if not self._profiles:
        raise ValueError("No profiles loaded. Use load_profile() first.")
    if X.ndim != 2 or X.size == 0:
        raise ValueError("Test matrix must be 2D and non-empty.")

    distances: Dict[str, float] = {}
    best_name: Optional[str] = None
    best_distance = float("inf")

    for name, entry in self._profiles.items():
        dists, _ = entry.verifier.score(X)
        avg_distance = float(dists.mean())
        distances[name] = avg_distance
        if avg_distance < best_distance:
            best_distance = avg_distance
            best_name = name

    if best_distance > unknown_threshold:
        best_name = None

    return best_name, best_distance, distances

def list_profiles(self):
    return list(self._profiles.keys())

def clear(self):
    self._profiles.clear()

def expected_feature_dim(self) -> Optional[int]:
    if not self._profiles:
        return None
    first = next(iter(self._profiles.values()))
    if first.verifier.mean is None:
        return None
    return int(first.verifier.mean.shape[0])

def iter_profiles(self):
    return list(self._profiles.values())

```

./keystroke_app/prompts.py

```
import random
from pathlib import Path
from typing import List, Sequence

def load_sentence_bank(path: Path) -> List[str]:
    sentences = [" ".join(line.strip().split()) for line in path.read_text(encoding="utf-8").splitlines]
    sentences = [s for s in sentences if s]
    if len(sentences) < 1000:
        raise ValueError(f"Expected at least 1000 sentences in {path}, found {len(sentences)}")
    return sentences

class PromptQueue:
    def __init__(self, sentence_bank: Sequence[str], size: int):
        if size < 1:
            raise ValueError("Prompt queue size must be >= 1.")
        self._sentence_bank = list(sentence_bank)
        if not self._sentence_bank:
            raise ValueError("Sentence bank cannot be empty.")
        self._size = size
        self._queue: List[str] = []
        self.reset()

    def _random_sentence(self) -> str:
        return random.choice(self._sentence_bank)

    def reset(self):
        self._queue = [self._random_sentence() for _ in range(self._size)]

    def current(self) -> str:
        if not self._queue:
            self.reset()
        return self._queue[0]

    def advance(self):
        if not self._queue:
            self.reset()
        return self._queue.pop(0)

    def as_text(self) -> str:
        return "\n".join(self._queue)
```

./keystroke_app/storage.py

```
from copy import deepcopy
import json
import time
from dataclasses import dataclass
```



```

from pathlib import Path
from typing import Any, Callable, Dict, List, Optional, Tuple

import numpy as np

from .capture import RAW_RUN_FORMAT, build_feature_vector_from_raw_run, normalize_raw_run
from .config import DEFAULT_DISTANCE_THRESHOLD, LEGACY_SCORE_THRESHOLD

@dataclass
class SessionData:
    enrollment_samples: List[np.ndarray]
    test_samples: List[np.ndarray]
    enrollment_raw_runs: List[Dict[str, Any]]
    test_raw_runs: List[Dict[str, Any]]
    enrollment_mean: Optional[np.ndarray]
    enrollment_inv_cov: Optional[np.ndarray]
    distance_threshold: float = DEFAULT_DISTANCE_THRESHOLD

FeatureBuilder = Callable[[Dict[str, Any]], np.ndarray]
_feature_builder: FeatureBuilder = build_feature_vector_from_raw_run

def set_feature_builder(builder: Optional[FeatureBuilder]) -> None:
    global _feature_builder
    _feature_builder = build_feature_vector_from_raw_run if builder is None else builder

def _build_feature_from_raw_run(run: Dict[str, Any]) -> np.ndarray:
    feat = _feature_builder(run)
    return np.asarray(feat, dtype=np.float32)

def _copy_samples(samples: List[np.ndarray]) -> List[np.ndarray]:
    return [np.array(s, dtype=np.float32, copy=True) for s in samples]

def _copy_raw_runs(raw_runs: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    return [deepcopy(run) for run in raw_runs]

def _serialize_raw_runs(raw_runs: List[Dict[str, Any]], field_name: str) -> List[Dict[str, Any]]:
    out: List[Dict[str, Any]] = []
    for i, run in enumerate(raw_runs):
        try:
            out.append(normalize_raw_run(run))
        except Exception as ex:
            raise ValueError(f"{field_name}[{i}] is invalid: {ex}") from ex
    return out

def _deserialize_raw_runs(raw: Any, field_name: str) -> Tuple[List[Dict[str, Any]], List[np.ndarray]]:
    if raw is None:

```

```

        raise ValueError(f"'{field_name}' is required.")
if not isinstance(raw, list):
    raise ValueError(f"'{field_name}' must be a list.")

out_runs: List[Dict[str, Any]] = []
out_features: List[np.ndarray] = []
for i, item in enumerate(raw):
    try:
        run = normalize_raw_run(item)
        feat = _build_feature_from_raw_run(run)
    except Exception as ex:
        raise ValueError(f"'{field_name} [{i}] is not a valid native key event run: {ex}") from ex
    out_runs.append(run)
    out_features.append(feat)
return out_runs, out_features

def _deserialize_vector(raw: Any, field_name: str) -> Optional[np.ndarray]:
    if raw is None:
        return None
    arr = np.asarray(raw, dtype=np.float32)
    if arr.ndim != 1 or arr.size == 0:
        raise ValueError(f"'{field_name}' must be a non-empty 1D numeric vector.")
    return arr

def _deserialize_matrix(raw: Any, field_name: str) -> Optional[np.ndarray]:
    if raw is None:
        return None
    arr = np.asarray(raw, dtype=np.float32)
    if arr.ndim != 2 or arr.shape[0] == 0 or arr.shape[1] == 0:
        raise ValueError(f"'{field_name}' must be a non-empty 2D numeric matrix.")
    return arr

def _feature_dim(data: SessionData) -> Optional[int]:
    if data.enrollment_mean is not None:
        return int(data.enrollment_mean.shape[0])
    if data.enrollment_samples:
        return int(data.enrollment_samples[0].shape[0])
    if data.test_samples:
        return int(data.test_samples[0].shape[0])
    return None

def _validate_dimensions(data: SessionData):
    dim = _feature_dim(data)
    if dim is None:
        return
    if data.enrollment_mean is not None and int(data.enrollment_mean.shape[0]) != dim:
        raise ValueError("Enrollment mean dimension does not match dataset dimension.")
    if data.enrollment_inv_cov is not None and data.enrollment_inv_cov.shape != (dim, dim):
        raise ValueError("Enrollment inverse covariance must be square and match dataset dimension.")
    for i, arr in enumerate(data.enrollment_samples):

```

```

        if int(arr.shape[0]) != dim:
            raise ValueError(f"Enrollment sample #{i + 1} has wrong feature dimension.")
    for i, arr in enumerate(data.test_samples):
        if int(arr.shape[0]) != dim:
            raise ValueError(f"Test sample #{i + 1} has wrong feature dimension.")
    for i, run in enumerate(data.enrollment_raw_runs):
        feat = _build_feature_from_raw_run(run)
        if int(feat.shape[0]) != dim:
            raise ValueError(f"Enrollment raw run #{i + 1} has wrong derived feature dimension.")
    for i, run in enumerate(data.test_raw_runs):
        feat = _build_feature_from_raw_run(run)
        if int(feat.shape[0]) != dim:
            raise ValueError(f"Test raw run #{i + 1} has wrong derived feature dimension.")

def _compute_enrollment_stats(
    enrollment_samples: List[np.ndarray],
) -> Tuple[Optional[np.ndarray], Optional[np.ndarray]]:
    if not enrollment_samples:
        return None, None

    X = np.stack(enrollment_samples, axis=0).astype(np.float32)
    mean = X.mean(axis=0)

    if X.shape[0] < 3:
        return mean, None

    cov = np.cov(X.T)
    cov = np.asarray(cov, dtype=np.float32)
    if cov.ndim == 0:
        cov = cov.reshape(1, 1)
    dim = X.shape[1]
    cov = cov + np.eye(dim, dtype=np.float32) * 1e-4
    inv_cov = np.linalg.inv(cov).astype(np.float32)
    return mean, inv_cov

def _normalize_distance_threshold(raw: Any) -> float:
    if not isinstance(raw, (int, float)):
        return float(DEFAULT_DISTANCE_THRESHOLD)
    value = float(raw)
    if value <= 0.0:
        return float(DEFAULT_DISTANCE_THRESHOLD)
    if abs(value - LEGACY_SCORE_THRESHOLD) < 1e-6:
        return float(DEFAULT_DISTANCE_THRESHOLD)
    return value

def session_data_to_payload(data: SessionData, dataset_version: int) -> Dict[str, Any]:
    _validate_dimensions(data)
    feature_dim = _feature_dim(data)
    enrollment_raw = _serialize_raw_runs(data.enrollment_raw_runs, "enrollment_raw_runs")
    test_raw = _serialize_raw_runs(data.test_raw_runs, "test_raw_runs")
    if len(data.enrollment_samples) != len(enrollment_raw):

```

```

        raise ValueError("Enrollment sample count must match enrollment raw run count.")
    if len(data.test_samples) != len(test_raw):
        raise ValueError("Test sample count must match test raw run count.")

payload: Dict[str, Any] = {
    "type": "keystroke_dataset",
    "version": int(dataset_version),
    "created_unix": int(time.time()),
    "run_format": RAW_RUN_FORMAT,
    "distance_threshold": float(data.distance_threshold),
    "feature_dim": int(feature_dim) if feature_dim is not None else None,
    "num_enrollment_runs": int(len(data.enrollment_samples)),
    "num_test_runs": int(len(data.test_samples)),
    "num_enrollment_raw_runs": int(len(enrollment_raw)),
    "num_test_raw_runs": int(len(test_raw)),
    "enrollment_runs": enrollment_raw,
    "test_runs": test_raw,
    "enrollment_mean": None if data.enrollment_mean is None else data.enrollment_mean.astype(float)
    "enrollment_inv_cov": None
    if data.enrollment_inv_cov is None
    else data.enrollment_inv_cov.astype(float).tolist(),
}
return payload

def payload_to_session_data(payload: Dict[str, Any]) -> SessionData:
    run_format = payload.get("run_format")
    if run_format != RAW_RUN_FORMAT:
        raise ValueError(
            f"Unsupported run_format '{run_format}'. Expected '{RAW_RUN_FORMAT}'."
        )

    enrollment_runs_raw = payload.get("enrollment_runs")
    test_runs_raw = payload.get("test_runs")
    enrollment_raw_runs, enrollment_samples = _deserialize_raw_runs(enrollment_runs_raw, "enrollment_runs")
    test_raw_runs, test_samples = _deserialize_raw_runs(test_runs_raw, "test_runs")

    enrollment_mean = _deserialize_vector(payload.get("enrollment_mean"), "enrollment_mean")
    if enrollment_mean is None and payload.get("reference") is not None:
        enrollment_mean = _deserialize_vector(payload.get("reference"), "reference")

    enrollment_inv_cov = _deserialize_matrix(payload.get("enrollment_inv_cov"), "enrollment_inv_cov")

    sample_dim: Optional[int] = None
    if enrollment_samples:
        sample_dim = int(enrollment_samples[0].shape[0])
    elif test_samples:
        sample_dim = int(test_samples[0].shape[0])
    if sample_dim is not None:
        if enrollment_mean is not None and int(enrollment_mean.shape[0]) != sample_dim:
            enrollment_mean = None
        if enrollment_inv_cov is not None and enrollment_inv_cov.shape != (sample_dim, sample_dim):
            enrollment_inv_cov = None

```

```

distance_threshold_raw = payload.get("distance_threshold", payload.get("score_threshold"))
distance_threshold = _normalize_distance_threshold(distance_threshold_raw)

data = SessionData(
    enrollment_samples=enrollment_samples,
    test_samples=test_samples,
    enrollment_raw_runs=enrollment_raw_runs,
    test_raw_runs=test_raw_runs,
    enrollment_mean=enrollment_mean,
    enrollment_inv_cov=enrollment_inv_cov,
    distance_threshold=float(distance_threshold),
)
_validate_dimensions(data)

feature_dim = payload.get("feature_dim")
if isinstance(feature_dim, int):
    dim = _feature_dim(data)
    if dim is not None and dim != feature_dim:
        # Keep loading when feature extraction backend differs from the dataset creator.
        pass

if data.enrollment_mean is None or data.enrollment_inv_cov is None:
    mean, inv_cov = _compute_enrollment_stats(data.enrollment_samples)
    if data.enrollment_mean is None:
        data.enrollment_mean = mean
    if data.enrollment_inv_cov is None:
        data.enrollment_inv_cov = inv_cov

return data

def save_session_data(path: Path, data: SessionData, dataset_version: int):
    payload = session_data_to_payload(data, dataset_version)
    path.write_text(json.dumps(payload, indent=2), encoding="utf-8")

def load_session_data(path: Path) -> SessionData:
    payload = json.loads(path.read_text(encoding="utf-8"))
    return payload_to_session_data(payload)

def merge_session_data(base: SessionData, incoming: SessionData) -> SessionData:
    merged_enrollment = _copy_samples(base.enrollment_samples) + _copy_samples(incoming.enrollment_samples)
    merged_test = _copy_samples(base.test_samples) + _copy_samples(incoming.test_samples)
    merged_enrollment_raw = _copy_raw_runs(base.enrollment_raw_runs) + _copy_raw_runs(incoming.enrollment_raw_runs)
    merged_test_raw = _copy_raw_runs(base.test_raw_runs) + _copy_raw_runs(incoming.test_raw_runs)

    if merged_enrollment:
        mean, inv_cov = _compute_enrollment_stats(merged_enrollment)
    else:
        mean = None
        inv_cov = None
        if incoming.enrollment_mean is not None:
            mean = np.array(incoming.enrollment_mean, dtype=np.float32, copy=True)

```

```

        elif base.enrollment_mean is not None:
            mean = np.array(base.enrollment_mean, dtype=np.float32, copy=True)
        if incoming.enrollment_inv_cov is not None:
            inv_cov = np.array(incoming.enrollment_inv_cov, dtype=np.float32, copy=True)
        elif base.enrollment_inv_cov is not None:
            inv_cov = np.array(base.enrollment_inv_cov, dtype=np.float32, copy=True)

merged_threshold = float(DEFAULT_DISTANCE_THRESHOLD)
merged = SessionData(
    enrollment_samples=merged_enrollment,
    test_samples=merged_test,
    enrollment_raw_runs=merged_enrollment_raw,
    test_raw_runs=merged_test_raw,
    enrollment_mean=mean,
    enrollment_inv_cov=inv_cov,
    distance_threshold=merged_threshold,
)
_validate_dimensions(merged)
return merged

def merge_session_data_file(path: Path, incoming: SessionData, dataset_version: int) -> SessionData:
    if path.exists():
        base = load_session_data(path)
    else:
        base = SessionData(
            enrollment_samples=[],
            test_samples=[],
            enrollment_raw_runs=[],
            test_raw_runs=[],
            enrollment_mean=None,
            enrollment_inv_cov=None,
            distance_threshold=4.5,
        )
    merged = merge_session_data(base, incoming)
    save_session_data(path, merged, dataset_version)
    return merged

```

./keystroke__app/verifier.py

```

from typing import Optional, Tuple

import numpy as np
from scipy.spatial.distance import mahalanobis

from .config import DEFAULT_DISTANCE_THRESHOLD

class Verifier:
    def __init__(self):
        self.mean: Optional[np.ndarray] = None
        self.inv_cov: Optional[np.ndarray] = None
        self.distance_threshold: float = float(DEFAULT_DISTANCE_THRESHOLD)

```

```

        self._n_enrollment_runs: int = 0

@property
def n_enrollment_runs(self) -> int:
    return self._n_enrollment_runs

def has_reference(self) -> bool:
    return self.mean is not None and self.inv_cov is not None

def clear(self):
    self.mean = None
    self.inv_cov = None
    self._n_enrollment_runs = 0

def fit(self, X: np.ndarray):
    if X.ndim != 2:
        raise ValueError("Enrollment data must be a 2D array shaped (n_runs, feature_dim).")
    n_runs, feature_dim = X.shape
    if n_runs < 3:
        raise ValueError("Need at least 3 enrollment runs to compute a stable covariance matrix.")

    mean = X.mean(axis=0).astype(np.float32)
    cov = np.cov(X.T)
    cov = np.asarray(cov, dtype=np.float32)
    if cov.ndim == 0: # handle single-feature edge case
        cov = cov.reshape(1, 1)
    regularization = np.eye(feature_dim, dtype=np.float32) * 1e-4
    cov = cov + regularization
    inv_cov = np.linalg.inv(cov).astype(np.float32)

    self.mean = mean
    self.inv_cov = inv_cov
    self._n_enrollment_runs = n_runs

def score(self, X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    if not self.has_reference():
        raise ValueError("Verifier has not been fitted; enroll first.")
    if X.ndim != 2:
        raise ValueError("Test data must be a 2D array shaped (n_runs, feature_dim).")
    if X.shape[1] != int(self.mean.shape[0]): # type: ignore[arg-type]
        raise ValueError(f"Feature dimension mismatch: got {X.shape[1]}, expected {self.mean.shape[0]}")

    distances = np.array(
        [float(mahalanobis(sample, self.mean, self.inv_cov)) for sample in X],
        dtype=np.float32,
    )
    inlier = distances <= self.distance_threshold
    return distances, inlier

```

./medium_english_sentences.txt

"A rolling stone gathers no moss" is a proverb.

"But don't you think that it's a little big?" asked the shopkeeper.

"He's a tiger when he's angry" is an example of metaphor.
 "How long does it take to get to Vienna on foot?" he inquired.
 "I can make it to my class on time," he thought.
 "I can't bear to be doing nothing!" you often hear people say.
 "I saw her five days ago," he said.
 "I want that book," he said to himself.
 "I'll be back in a minute," he added.
 "I'm the happiest man in the world," Tom said to himself.
 "Superman" is showing at the movie theater this month.
 "Thank you, I'd love to have another piece of cake," said the shy young man.
 "The good die young" is an old saying which may or may not be true.
 "What should I do?" I said to myself.
 "What will you have to do?" asked her friend.
 1980 was the year that I was born.
 67% of those who never smoked said they worried about the health effects of passive smoking.
 A "renovator's dream" in real estate parlance generally means that the place is a real dump.
 A 5% consumption tax is levied on purchases of most goods and services.
 A 6% yield is guaranteed on the investment.
 A baby has no knowledge of good and evil.
 A baby is incapable of taking care of itself.
 A bad cold has kept me from studying this week.
 A bad cold prevented her from attending the class.
 A bad habit, once formed, is difficult to get rid of.
 A bad writer's prose is full of hackneyed phrases.
 A bat flying in the sky looks like a butterfly.
 A bat hunts food and eats at night, but sleeps during the day.
 A bat is no more a bird than a rat is.
 A bead of sweat started forming on his brow.
 A beam of sunlight came through the clouds.
 A bear will not touch a dead body.
 A beautiful lake lies just beyond the forest.
 A beautiful salesgirl waited on me in the shop.
 A beautiful woman was seated one row in front of me.
 A belt keeps your pants from falling down.
 A bicycle will rust if you leave it in the rain.
 A big bomb fell, and a great many people lost their lives.
 A big bridge was built over the river.
 A big surprise was waiting for me at home.
 A big wave swept the man off the boat.
 A bird can glide through the air without moving its wings.
 A bird in the hand is better than two in the bush.
 A bird in the hand is worth two in the bush.
 A bird is known by its song and a man by his way of talking.
 A bird was flying high up in the sky.
 A blast of cold air swept through the house.
 A blender lets you mix different foods together.
 A blind person's hearing is often very acute.
 A boat suddenly appeared out of the mist.

./owen.json

```
{
  "type": "keystroke_dataset",
```



```

"version": 3,
"created_unix": 1772319842,
"run_format": "native_key_events_v1",
"distance_threshold": 4.5,
"feature_dim": 19,
"num_enrollment_runs": 300,
"num_test_runs": 0,
"num_enrollment_raw_runs": 300,
"num_test_raw_runs": 0,
"enrollment_runs": [
  {
    "events": [
      {
        "type": "keydown",
        "timestamp_ms": 1772314990392.0,
        "keycode": 943782142,
        "keysym": "Shift_L",
        "char": ""
      },
      {
        "type": "keydown",
        "timestamp_ms": 1772314990644.0,
        "keycode": 205520977,
        "keysym": "Q",
        "char": "Q"
      },
      {
        "type": "keydown",
        "timestamp_ms": 1772314990712.0,
        "keycode": 222298199,
        "keysym": "W",
        "char": "W"
      },
      {
        "type": "keydown",
        "timestamp_ms": 1772314991137.0,
        "keycode": 855638143,
        "keysym": "BackSpace",
        "char": ""
      },
      {
        "type": "keyup",
        "timestamp_ms": 1772314991176.0,
        "keycode": 855638143,
        "keysym": "BackSpace",
        "char": ""
      },
      {

```

./sambhu.json

```

{
  "type": "keystroke_dataset",

```

```

"version": 3,
"created_unix": 1772322727,
"run_format": "native_key_events_v1",
"distance_threshold": 4.5,
"feature_dim": 19,
"num_enrollment_runs": 310,
"num_test_runs": 0,
"num_enrollment_raw_runs": 310,
"num_test_raw_runs": 0,
"enrollment_runs": [
  {
    "events": [
      {
        "type": "keydown",
        "timestamp_ms": 57016756.0,
        "keycode": 62,
        "keysym": "Shift_R",
        "char": ""
      },
      {
        "type": "keydown",
        "timestamp_ms": 57016945.0,
        "keycode": 43,
        "keysym": "H",
        "char": "H"
      },
      {
        "type": "keyup",
        "timestamp_ms": 57017012.0,
        "keycode": 62,
        "keysym": "Shift_R",
        "char": ""
      },
      {
        "type": "keyup",
        "timestamp_ms": 57017048.0,
        "keycode": 43,
        "keysym": "h",
        "char": ""
      },
      {
        "type": "keydown",
        "timestamp_ms": 57017089.0,
        "keycode": 26,
        "keysym": "e",
        "char": "e"
      },
      {

```

./test_verifier.py

```
import numpy as np
```

```

from keystroke_app.verifier import Verifier

def main():
    rng = np.random.default_rng(42)
    feature_dim = 19
    true_mean = np.linspace(-0.5, 0.5, feature_dim, dtype=np.float32)

    enrollment = true_mean + rng.normal(scale=0.5, size=(30, feature_dim)).astype(np.float32)

    verifier = Verifier()
    verifier.fit(enrollment)

    genuine = true_mean + rng.normal(scale=0.5, size=(3, feature_dim)).astype(np.float32)
    impostor = (true_mean + 2.0) + rng.normal(scale=0.5, size=(3, feature_dim)).astype(np.float32)

    samples = np.vstack([genuine, impostor]).astype(np.float32)
    distances, _ = verifier.score(samples)

    print("Distances (first 3 genuine, last 3 impostor):")
    for idx, dist in enumerate(distances):
        label = "genuine" if idx < 3 else "impostor"
        print(f" Sample {idx + 1} ({label}): {dist:.4f}")

    threshold = (distances[:3].max() + distances[3:].min()) / 2.0
    verifier.distance_threshold = float(threshold)
    _, inlier = verifier.score(samples)

    assert np.all(inlier[:3]), "Expected genuine samples to be inliers"
    assert not np.any(inlier[3:]), "Expected impostor samples to be outliers"

    try:
        verifier.fit(true_mean.reshape(1, -1))
    except ValueError:
        pass
    else:
        raise AssertionError("fit should raise ValueError for fewer than 3 runs")

if __name__ == "__main__":
    main()

```

./test.py

```

from keystroke_app.app import main

if __name__ == "__main__":
    main()

```

./threshold_tuner.py

```
import argparse
from pathlib import Path
from typing import Optional

import numpy as np

from keystroke_app.storage import load_session_data
from keystroke_app.verifier import Verifier


def describe_distances(label: str, distances: np.ndarray):
    print(f"{label} distances:")
    print(f"  count = {distances.size}")
    print(f"  min   = {distances.min():.4f}")
    print(f"  max   = {distances.max():.4f}")
    print(f"  mean  = {distances.mean():.4f}")


def main():
    parser = argparse.ArgumentParser(description="Suggest Mahalanobis distance thresholds from a dataset")
    parser.add_argument("dataset", type=Path, help="Path to keystroke_dataset.json")
    args = parser.parse_args()

    data = load_session_data(args.dataset)
    if len(data.enrollment_samples) < 3:
        raise SystemExit("Dataset needs at least 3 enrollment runs to tune a threshold.")

    verifier = Verifier()
    enrollment_matrix = np.stack(data.enrollment_samples, axis=0)
    verifier.fit(enrollment_matrix)

    enroll_distances, _ = verifier.score(enrollment_matrix)
    describe_distances("Enrollment", enroll_distances)

    test_distances: Optional[np.ndarray] = None
    if data.test_samples:
        test_matrix = np.stack(data.test_samples, axis=0)
        test_distances, _ = verifier.score(test_matrix)
        describe_distances("Test", test_distances)
    else:
        print("No test samples in dataset; cannot suggest threshold from test data.")

    if test_distances is not None:
        max_enroll = float(enroll_distances.max())
        min_test = float(test_distances.min())
        suggested = (max_enroll + min_test) / 2.0
        print(f"\nSuggested threshold: {suggested:.4f}")
        if max_enroll > min_test:
            print("WARNING: Enrollment and test distance distributions overlap; consider collecting more")
    else:
        # fallback suggestion using enrollment only (e.g., 1 std above max)
        fallback = float(enroll_distances.max()) + enroll_distances.std()
```

```
print(f"\nSuggested threshold (fallback): {fallback:.4f}")

if __name__ == "__main__":
    main()
```